



Desarrollo Backend II

EFT

Nombre: Valeria Acevedo Cabrera

Profesor: Marcelo Zepeda

Analista Programador Computacional

Fecha: 19-07-2026

Índice

Introducción	4
Captura de la Estructura general del proyecto MiniMarket Plus	5
Descripción general del proyecto	6
Desarrollo de Servicio REST	7
Implementación de seguridad.....	19
Spring Security	20
Autenticación mediante JWT	21
Autorización basada en roles	23
Filtro de autenticación JWT	25
Pruebas unitarias.....	27
Organización de las pruebas.....	29
Implementación de las pruebas.....	30
Ejecución de las pruebas.....	31
Cobertura de código	33
Resultados obtenidos	34
Reporte final de cobertura generado por JaCoCo	36
Documentación de la API con OpenAPI y HATEOAS.....	38
Implementación de HATEOAS mediante Model Assemblers.....	39
Documentación de la API	41
OpenApi (Swagger)	42
Spring HATEOAS	42
Estandarización de respuestas HTTP	55
Manejo global de excepciones	55
Validación de la documentación	56

Exportación del documento OpenAPI desde el endpoint /v3/api-docs.	56
Importación del documento OpenAPI en Postman para validar los endpoints documentados.	57
Integración del backend.....	58
Mejoras implementadas durante el desarrollo	59
Conclusión.....	60

Introducción

MiniMarket Plus consiste en el desarrollo de una API REST orientada a la gestión de un minimarket, permitiendo administrar productos, categorías, inventario, usuarios, roles, ventas y carritos de compra mediante una arquitectura backend basada en Spring Boot. El proyecto fue desarrollado siguiendo una arquitectura por capas, separando claramente las responsabilidades entre controladores, servicios, repositorios y entidades, favoreciendo la mantenibilidad, escalabilidad y reutilización del código.

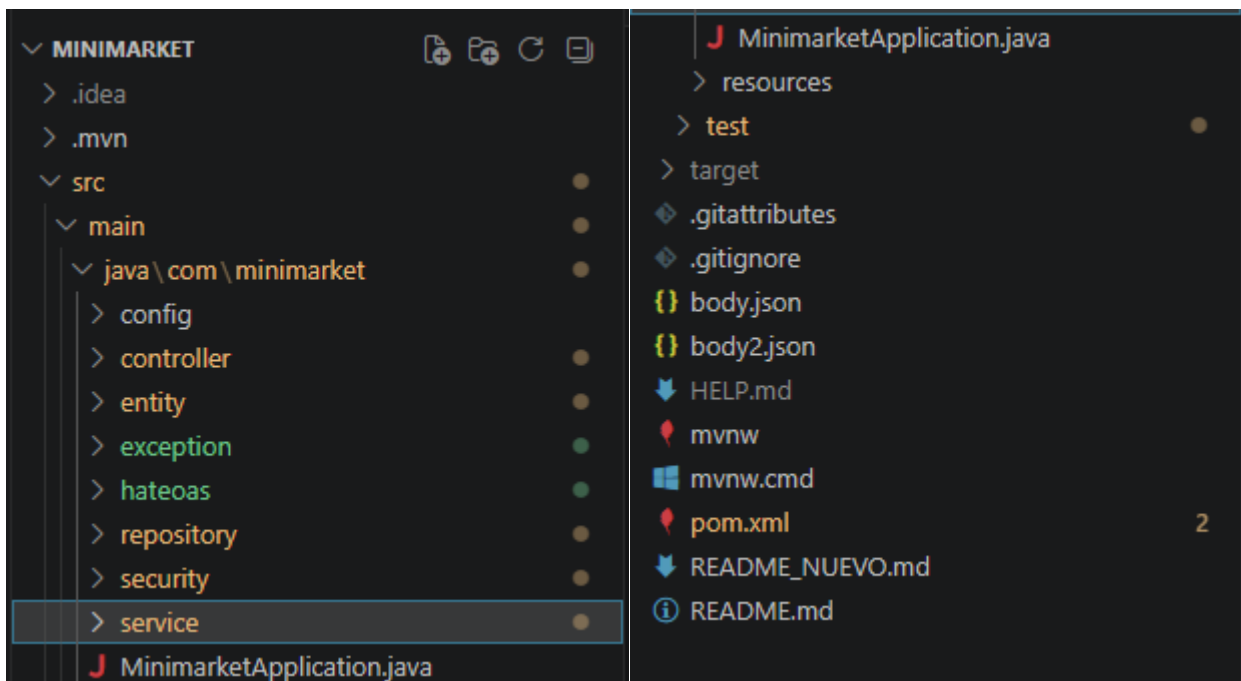
Durante el desarrollo del proyecto se incorporaron progresivamente distintos componentes tecnológicos. En una primera etapa se implementó la estructura base del backend junto con las operaciones CRUD necesarias para administrar las principales entidades del sistema. Posteriormente se integró un mecanismo de autenticación y autorización utilizando Spring Security y JSON Web Token (JWT), permitiendo proteger los recursos de la aplicación mediante roles específicos para administradores, empleados y clientes. Finalmente, se documentó completamente la API utilizando OpenAPI (Swagger) y se implementó Spring HATEOAS para enriquecer las respuestas REST mediante enlaces dinámicos entre recursos.

Como complemento a las funcionalidades implementadas, se desarrolló un conjunto de pruebas unitarias utilizando JUnit 5, Mockito y JaCoCo, verificando el correcto funcionamiento de la lógica de negocio y evaluando la cobertura del código. Estas pruebas permitieron detectar oportunidades de mejora durante el desarrollo, fortaleciendo la estabilidad y confiabilidad del sistema.

El presente documento reúne la arquitectura desarrollada, las principales decisiones técnicas adoptadas, la implementación de los mecanismos de seguridad, la documentación de los servicios REST, las pruebas realizadas y las evidencias obtenidas durante la construcción del backend, demostrando la integración de todos los contenidos abordados durante la asignatura.

Captura de la Estructura general del proyecto MiniMarket Plus

La imagen presenta la estructura actualizada del proyecto MiniMarket Plus. Además de los paquetes correspondientes a controladores, servicios, repositorios, entidades y seguridad, se incorporaron los paquetes hateoas y exception. El paquete hateoas centraliza la construcción de enlaces hipermedia mediante Model Assemblers, mientras que el paquete exception contiene las clases responsables del manejo global y estandarizado de errores.

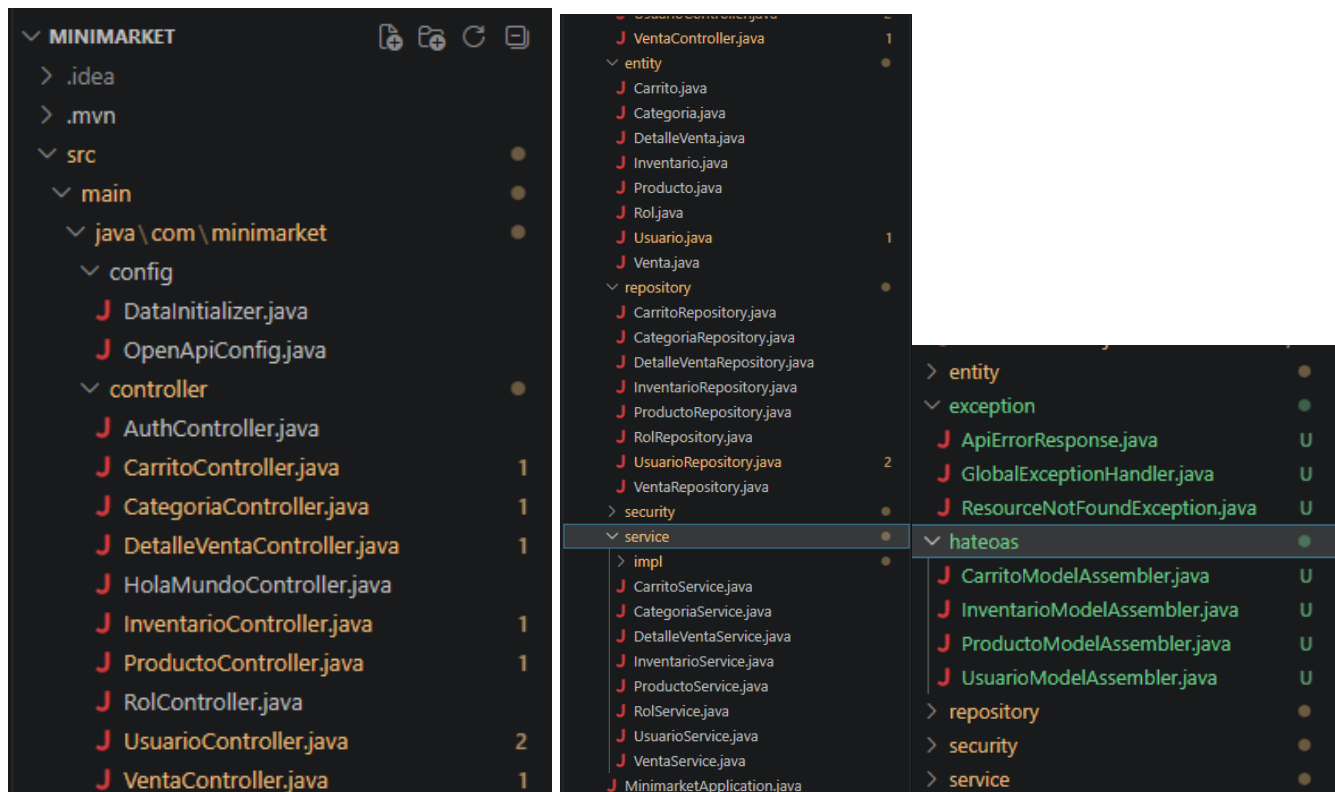


Descripción general del proyecto

MiniMarket Plus corresponde al desarrollo de una API REST destinada a apoyar la gestión operativa de una cadena de minimarkets. El sistema permite administrar productos, categorías, inventario, usuarios, roles, ventas y carritos de compra mediante servicios REST protegidos con autenticación basada en JWT.

La aplicación fue desarrollada utilizando Java 17 y Spring Boot, incorporando Spring Data JPA para el acceso a datos, Spring Security para la protección de los recursos, H2 Database como base de datos de desarrollo, OpenAPI para la documentación automática de la API y Spring HATEOAS para mejorar la navegación entre recursos.

La solución implementa una arquitectura desacoplada basada en capas, permitiendo separar la lógica de negocio, el acceso a datos y la exposición de servicios REST, facilitando futuras ampliaciones del sistema.



Desarrollo de Servicio REST

En el proyecto MiniMarket Plus, se desarrollaron distintos servicios REST encargados de administrar las principales entidades del sistema. Cada servicio fue implementado mediante controladores Spring Boot, utilizando operaciones CRUD para permitir la creación, consulta, actualización y eliminación de información.

La lógica de negocio fue implementada en la capa de servicios, mientras que el acceso a los datos se realizó mediante Spring Data JPA, manteniendo una separación clara de responsabilidades entre las distintas capas de la aplicación.

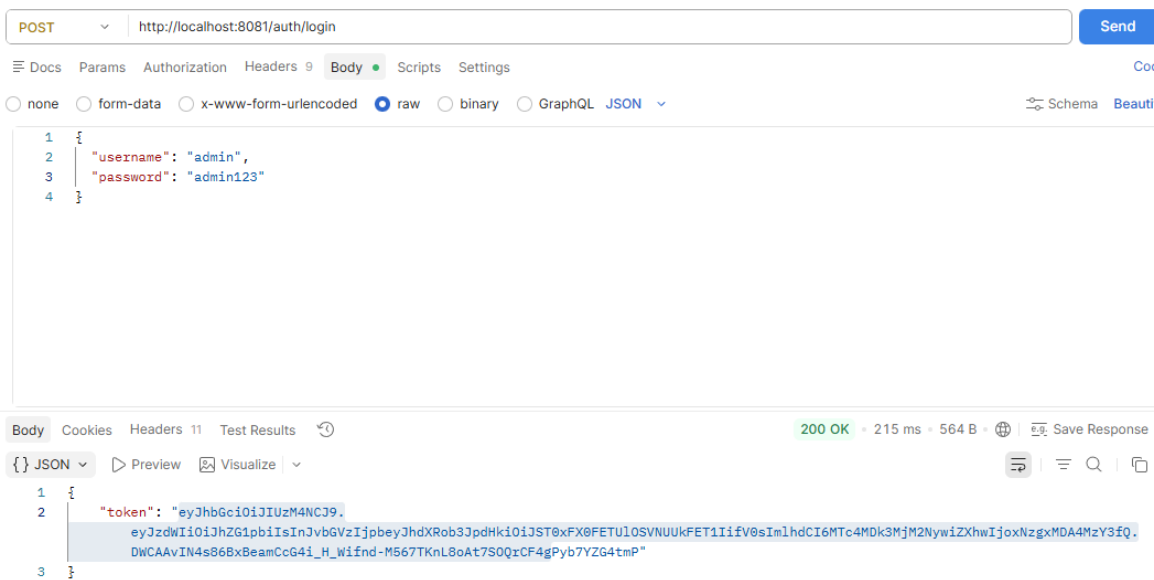
AuthController

El controlador de autenticación permite validar las credenciales de los usuarios y generar un token JWT, el cual será utilizado para acceder a los recursos protegidos de la aplicación.

Quando un usuario inicia sesión correctamente, el sistema genera un token firmado que posteriormente debe enviarse en el encabezado Authorization de cada solicitud protegida.

Captura de Generación de token JWT mediante el endpoint de autenticación.

La imagen muestra la autenticación correcta de un usuario mediante el endpoint `/auth/login`, obteniendo un token JWT válido para acceder a los recursos protegidos del sistema.



ProductoController

El servicio REST de productos permite administrar el catálogo de productos disponible en el sistema. Se implementaron operaciones para listar, consultar, registrar, modificar y eliminar productos.

Los endpoints fueron documentados utilizando OpenAPI y protegidos mediante Spring Security según los permisos definidos para cada rol.

Captura de acceso autorizado mediante token valido

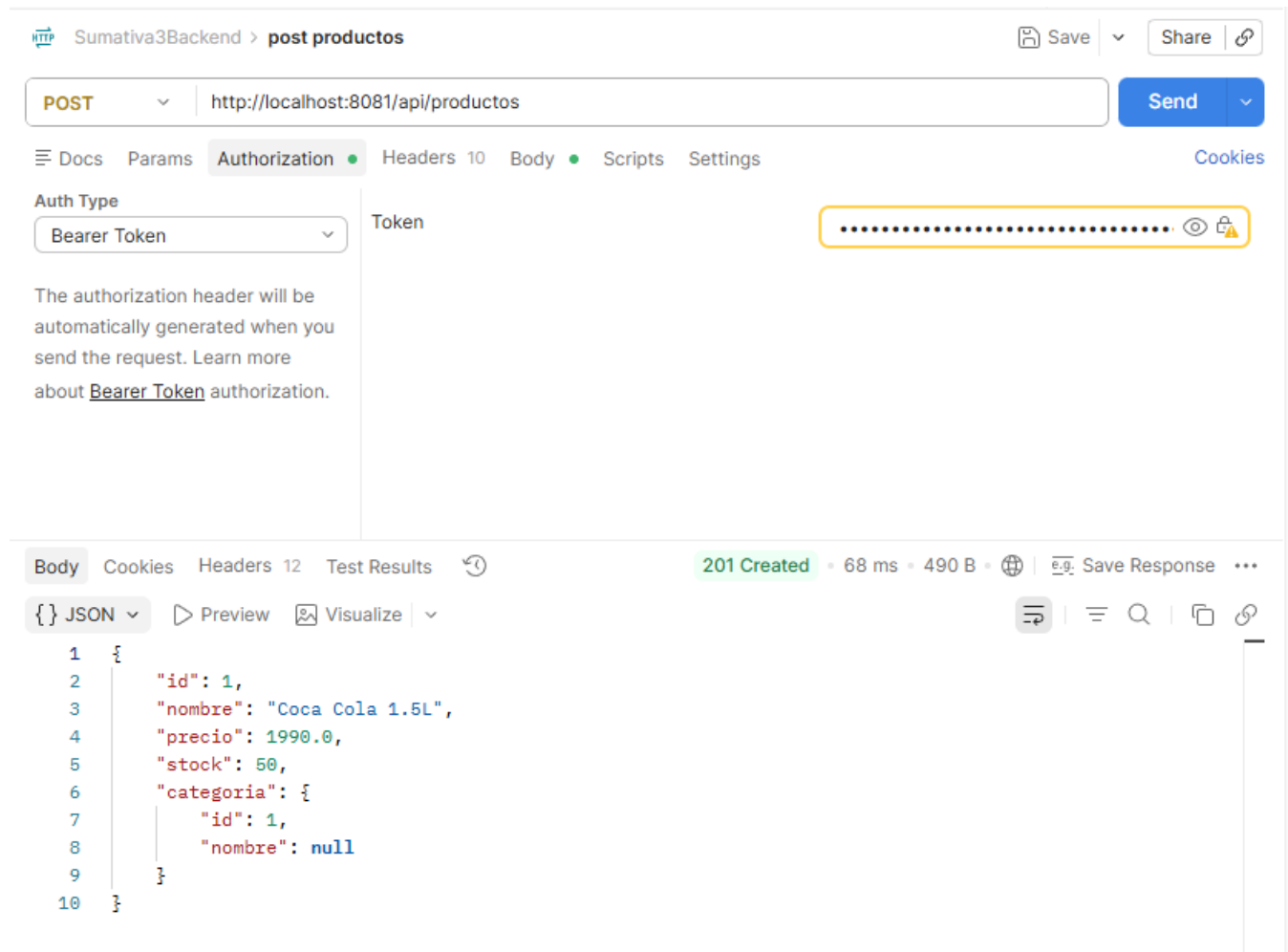
GET api/productos

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8081/api/productos
- Auth Type:** Bearer ...
- Token:** [Redacted]
- Body:** [Redacted]
- Test Results:** 200 OK • 44 ms • 37.73 KB
- JSON Response:**

```
1 [
2   {
3     "id": 1,
4     "nombre": "Coca Cola 2L",
5     "precio": 2500.0,
6     "stock": 100,
```


POST api/producto.

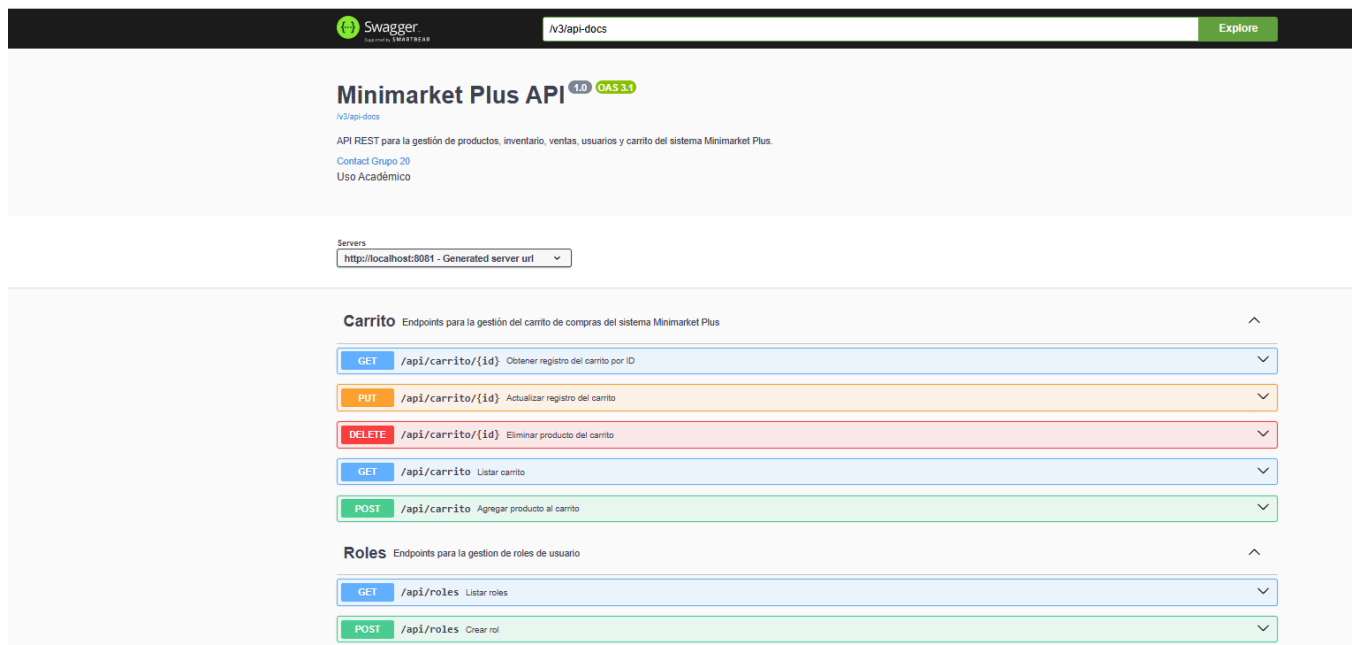


The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8081/api/productos`
- Method:** `POST`
- Auth Type:** `Bearer Token`
- Token:** A masked token represented by dots.
- Response:** `201 Created` with a status of `68 ms` and a body size of `490 B`.
- Body:** A JSON object representing a product:

```
1 {
2   "id": 1,
3   "nombre": "Coca Cola 1.5L",
4   "precio": 1990.0,
5   "stock": 50,
6   "categoria": {
7     "id": 1,
8     "nombre": null
9   }
10 }
```

- **Swagger del controlador**

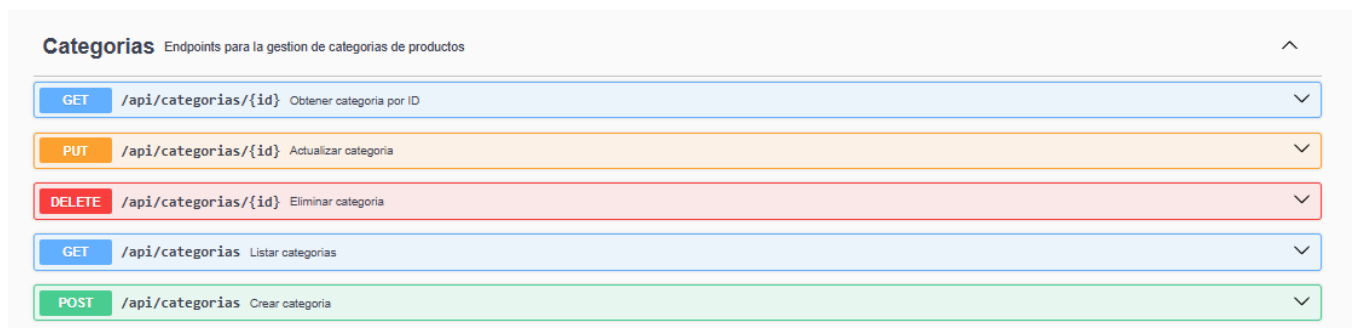


CategoriaController

Este controlador permite administrar las categorías utilizadas para clasificar los productos del sistema.

Se implementaron operaciones CRUD que permiten mantener la información organizada y facilitar la búsqueda de productos por categoría.

Captura del CRUD de categorías

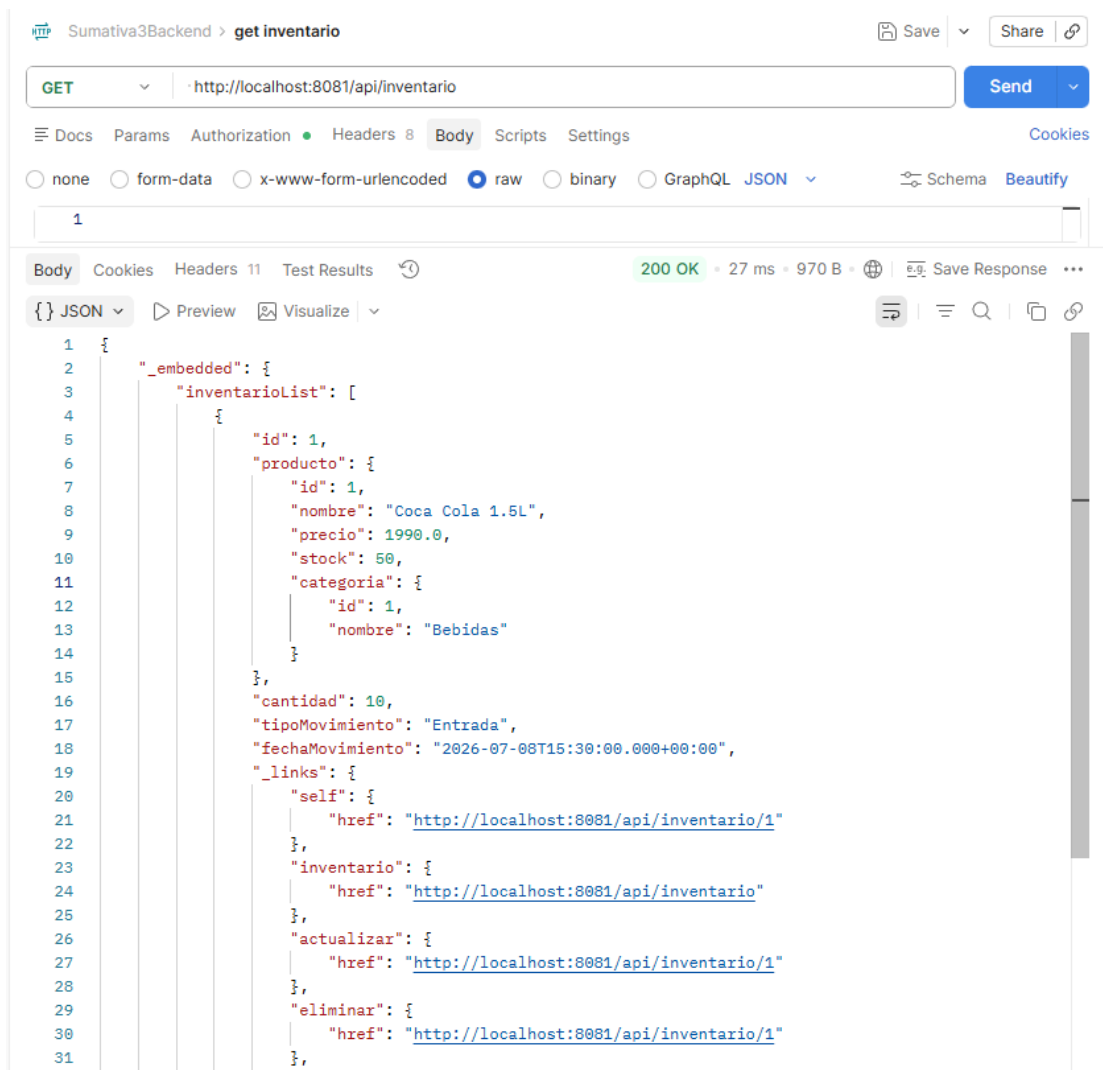


InventarioController

El controlador de inventario administra los movimientos de entrada y salida de productos, permitiendo mantener actualizado el stock disponible.

Las operaciones implementadas consideran la validación de permisos antes de registrar modificaciones sobre el inventario.

Movimiento de inventario desde Postman.



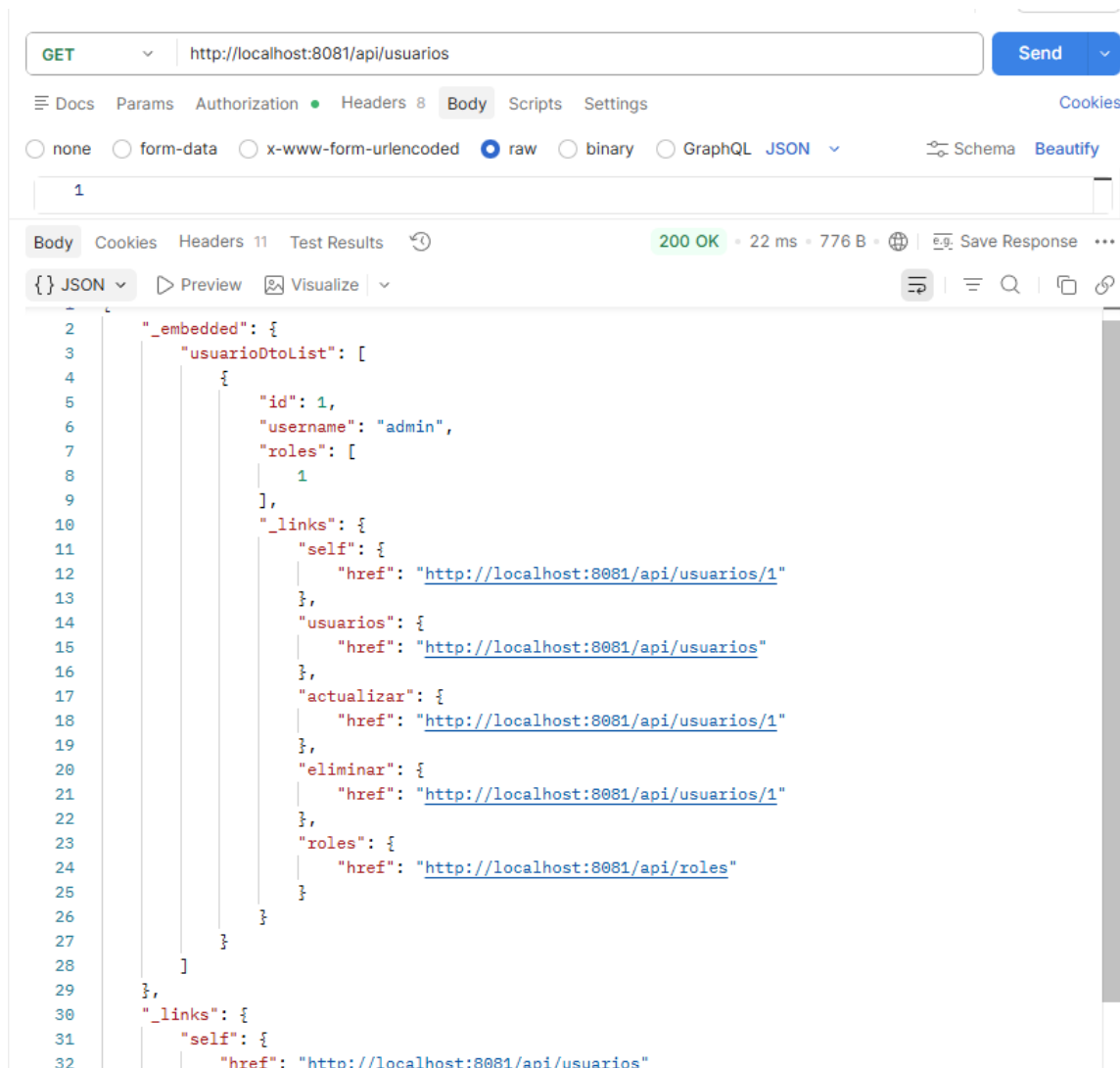
UsuarioController

Este controlador permite administrar los usuarios registrados dentro del sistema.

Las operaciones disponibles consideran la creación, consulta y actualización de usuarios, manteniendo protegida la información sensible mediante Spring Security.

Evidencia

Listado de usuarios



The screenshot shows a REST client interface with a GET request to `http://localhost:8081/api/usuarios`. The response is a 200 OK status with a response time of 22 ms and a body size of 776 B. The response body is displayed in JSON format, showing a list of users. The first user in the list is an admin user with id 1.

```

1
2  "_embedded": {
3    "usuarioDtoList": [
4      {
5        "id": 1,
6        "username": "admin",
7        "roles": [
8          1
9        ],
10       "_links": {
11         "self": {
12           "href": "http://localhost:8081/api/usuarios/1"
13         },
14         "usuarios": {
15           "href": "http://localhost:8081/api/usuarios"
16         },
17         "actualizar": {
18           "href": "http://localhost:8081/api/usuarios/1"
19         },
20         "eliminar": {
21           "href": "http://localhost:8081/api/usuarios/1"
22         },
23         "roles": {
24           "href": "http://localhost:8081/api/roles"
25         }
26       }
27     }
28   ],
29   "_links": {
30     "self": {
31       "href": "http://localhost:8081/api/usuarios"
32     }

```

RolController

El controlador de roles administra los perfiles utilizados para controlar los permisos de acceso a la aplicación.

Se implementaron restricciones que permiten que únicamente usuarios con privilegios de administrador puedan modificar esta información.

Captura de Intento exitoso con administrador.

The screenshot displays a REST client interface. At the top, a dropdown menu shows 'POST' and the URL 'http://localhost:8081/api/roles'. A 'Send' button is visible. Below the URL bar, tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', and 'Settings' are shown. The 'Body' tab is selected, and the 'raw' radio button is chosen. The body content is a JSON object:

```
1 {
2   "nombre": "EMPLEADO"
3 }
```

 Below the body tab, a status bar indicates '201 Created' with a response time of '13 ms' and a size of '418 B'. A 'Save Response' button is present. The response body is shown in a JSON format:

```
1 {
2   "id": 2,
3   "nombre": "EMPLEADO"
4 }
```

Captura de Intento rechazado con empleado.

Sumativa3Backend > post usuarios Save Share

POST ▼ Send ▼

Docs Params Authorization Headers 10 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```

1  {
2    "username": "empleado",
3    "password": "empleado123",
4    "roles": [
5      {
6        "id": 2
7      }
8    ]
9  }

```

Body Cookies Headers 12 Test Results ↺ 201 Created • 86 ms • 456 B • 🌐 🔗 Save Response ⋮

{} **JSON** ▼ ▶ Preview 🖼 Visualize ▼ ↺ ≡ 🔍 📄 🔗

```

1  {
2    "id": 2,
3    "username": "empleado",
4    "roles": [
5      {
6        "id": 2,
7        "nombre": null
8      }
9    ]
10 }

```

Sumativa3Backend > login Copy Save Share

POST http://localhost:8081/auth/login Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```

1 {
2   "username": "empleado",
3   "password": "empleado123"
4 }

```

Body Cookies Headers 11 Test Results 200 OK · 76 ms · 515 B · Save Response

JSON Preview Visualize

```

1 {
2   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlbXBsZWZkbyIsInJvbGVzIjpbIkpvaGVudXExFURPIl0sIm1hdCI6MTc4NDQ0ODU0OSwiZXhwIjoxNzg0NDg0NTQ5fQ.WLy1Tz_HPE8wWMIMvGbPzG8pKEgwKcT6WHJ8tp4YHw0"
3 }

```

Sumativa3Backend > rol Save Share

POST http://localhost:8081/api/roles Send

Docs Params **Authorization** Headers 10 **Body** Scripts Settings Cookies

Auth Type Bearer Token Token MvGbPzG8pKEgwKcT6WHJ8tp4YHw0

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 10 Test Results 403 Forbidden · 6 ms · 306 B · Save Response

Raw Preview Try with different auth credentials

```

1

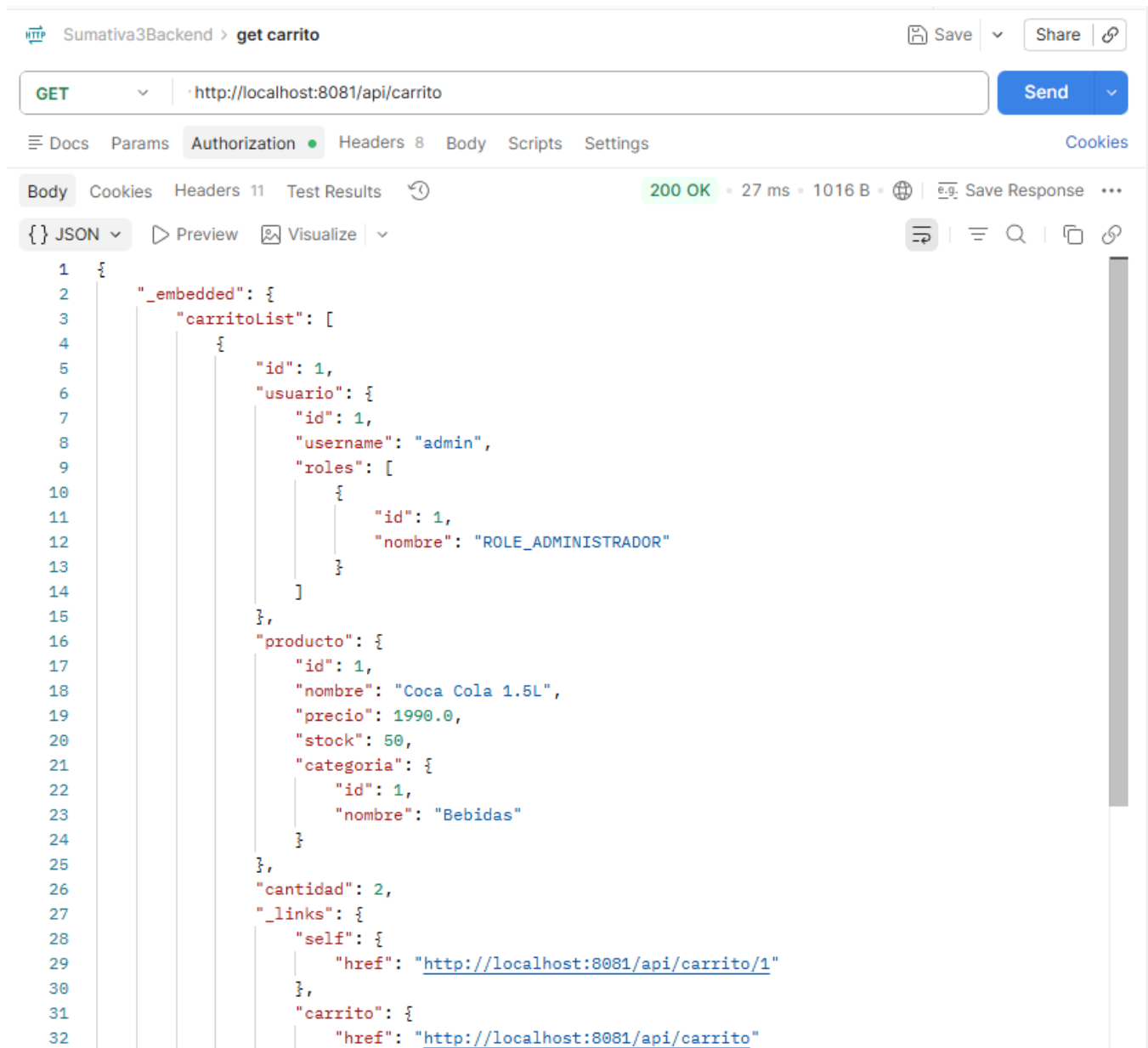
```

CarritoController

El servicio REST de carrito permite registrar los productos seleccionados por un usuario antes de generar una venta.

Se implementaron operaciones para agregar, consultar y eliminar productos asociados al carrito de compras.

Captura de Listado del carrito



Sumativa3Backend > get carrito

GET http://localhost:8081/api/carrito

200 OK • 27 ms • 1016 B • Save Response

```

1  {
2    "_embedded": {
3      "carritoList": [
4        {
5          "id": 1,
6          "usuario": {
7            "id": 1,
8            "username": "admin",
9            "roles": [
10             {
11               "id": 1,
12               "nombre": "ROLE_ADMINISTRADOR"
13             }
14           ]
15         },
16         "producto": {
17           "id": 1,
18           "nombre": "Coca Cola 1.5L",
19           "precio": 1990.0,
20           "stock": 50,
21           "categoria": {
22             "id": 1,
23             "nombre": "Bebidas"
24           }
25         },
26         "cantidad": 2,
27         "_links": {
28           "self": {
29             "href": "http://localhost:8081/api/carrito/1"
30           },
31           "carrito": {
32             "href": "http://localhost:8081/api/carrito"

```



```
31     "carrito": {
32       "href": "http://localhost:8081/api/carrito"
33     },
34     "actualizar": {
35       "href": "http://localhost:8081/api/carrito/1"
36     },
37     "eliminar": {
38       "href": "http://localhost:8081/api/carrito/1"
39     },
40     "usuario": {
41       "href": "http://localhost:8081/api/usuarios/1"
42     },
43     "producto": {
44       "href": "http://localhost:8081/api/productos/1"
45     }
46   }
47 }
48 ]
49 },
50 "_links": {
51   "self": {
52     "href": "http://localhost:8081/api/carrito"
53   }
54 }
55 }
```

VentaController

El controlador de ventas administra el registro de las compras realizadas por los usuarios. Cada venta almacena la información general de la transacción y mantiene la relación con los productos adquiridos mediante la entidad DetalleVenta.

Captura de Registro de venta.

The screenshot displays a REST client interface with a POST request to `http://localhost:8081/api/ventas`. The request body is a JSON object:

```
1 {
2   "usuario": {
3     "id": 1
4   },
5   "fecha": "2026-07-17T18:30:00.000Z"
6 }
```

The response is a `201 Created` status with a response time of 11 ms and a body size of 503 B. The response body is shown in JSON format:

```
1 {
2   "id": 1,
3   "usuario": {
4     "id": 1,
5     "username": null,
6     "roles": null
7   },
8   "fecha": "2026-07-17T18:30:00.000+00:00",
9   "detalles": null
10 }
```

Implementación de seguridad

Con el objetivo de proteger la información del sistema y restringir el acceso a las operaciones críticas, se implementó un mecanismo de seguridad utilizando Spring Security junto con JSON Web Token (JWT). Esta configuración permite autenticar a los usuarios y controlar el acceso a los distintos endpoints según el rol asignado.

La aplicación fue configurada con una política Stateless, por lo que el servidor no mantiene sesiones activas. Cada solicitud realizada a un recurso protegido debe incluir un token JWT válido en el encabezado Authorization, el cual es validado antes de procesar la petición.

Para ello, se agregaron las dependencias necesarias al proyecto, entre las cuales destacan Spring Security para la gestión de autenticación y autorización, Spring Web para la exposición de servicios REST y la biblioteca JWT para la generación y validación de tokens JWT.

Evidencia de pom.xml

Spring Security: gestión de autenticación y autorización

```
36         </dependency>
37         <dependency>
38             <groupId>org.springframework.boot</groupId>
39             <artifactId>spring-boot-starter-security</artifactId>
40         </dependency>
41         <dependency>
```

La dependencia spring-boot-starter-security proporciona los componentes necesarios para implementar autenticación, autorización, filtros de seguridad y control de acceso a los endpoints de la API.

```
<dependency>
|   <groupId>org.springframework.boot</groupId>
|   <artifactId>spring-boot-starter-web</artifactId>
| </dependency>
```

La incorporación de estas dependencias permitió construir una arquitectura de seguridad orientada a servicios REST, compatible con una estrategia Stateless y preparada para futuras ampliaciones del sistema.

Spring Security

Spring Security fue utilizado para definir las reglas de acceso de la aplicación. A través de la clase SecurityConfig se configuraron los endpoints públicos y protegidos, estableciendo permisos específicos para cada rol del sistema.

Además, se deshabilitó CSRF debido a que la aplicación corresponde a una API REST y se configuró el manejo de sesiones como STATELESS, permitiendo que la autenticación dependa únicamente del token JWT.

Captura de clase SecurityConfig.java

```
17 @Configuration
18 @EnableMethodSecurity
19 public class SecurityConfig {
20
21     private final JwtAuthenticationFilter jwtAuthenticationFilter;
22
23     public SecurityConfig(CustomUserDetailsService customUserDetailsService,
24                         JwtAuthenticationFilter jwtAuthenticationFilter) {
25         this.jwtAuthenticationFilter = jwtAuthenticationFilter;
26     }
27
28     @Bean
29     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
30         http
31             .csrf(csrf -> csrf.disable())
32             .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
33             .sessionManagement(session ->
34                 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
35             )
36             .authorizeHttpRequests(auth -> auth
37                 .requestMatchers(...patterns: "/auth/login").permitAll()
38                 .requestMatchers(...patterns: "/h2-console/**").permitAll()
39                 .requestMatchers(...patterns: "/public/**").permitAll()
40
41                 // Swagger / OpenAPI
42                 .requestMatchers(...patterns: "/swagger-ui/**").permitAll()
43                 .requestMatchers(...patterns: "/swagger-ui.html").permitAll()
44                 .requestMatchers(...patterns: "/v3/api-docs/**").permitAll()
45
46                 .requestMatchers(...patterns: "/api/usuarios").hasAuthority(authority: "ROLE_ADMINISTRADOR")
47                 .requestMatchers(...patterns: "/api/usuarios/**").hasAuthority(authority: "ROLE_ADMINISTRADOR")
48                 .requestMatchers(...patterns: "/api/productos/**").hasAnyAuthority(...authorities: "ROLE_CLIENTE", "ROLE_EMPLEADO", "ROLE_ADMINISTRADOR")
49                 .requestMatchers(...patterns: "/api/inventario/**").hasAnyAuthority(...authorities: "ROLE_EMPLEADO", "ROLE_ADMINISTRADOR")
50                 .requestMatchers(...patterns: "/api/ventas/**").hasAnyAuthority(...authorities: "ROLE_EMPLEADO", "ROLE_ADMINISTRADOR")
51                 .requestMatchers(...patterns: "/api/categorias").hasAnyAuthority(...authorities: "ROLE_ADMINISTRADOR")
52                 .requestMatchers(...patterns: "/api/categorias/**").hasAnyAuthority(...authorities: "ROLE_ADMINISTRADOR")
53                 .requestMatchers(...patterns: "/api/carrito/**").hasAnyAuthority(...authorities: "ROLE_EMPLEADO", "ROLE_CLIENTE", "ROLE_ADMINISTRADOR")
54                 .requestMatchers(...patterns: "/api/detalle-ventas/**").hasAnyAuthority(...authorities: "ROLE_EMPLEADO", "ROLE_ADMINISTRADOR")
55                 .requestMatchers(...patterns: "/api/roles/**").hasAuthority(authority: "ROLE_ADMINISTRADOR")
56
57                 .anyRequest().authenticated()
58             )
59             .addFilterBefore(jwtAuthenticationFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);
60
61     return http.build();
62 }
```

```
63
64  @Bean
65  public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig)
66      throws Exception {
67      return authConfig.getAuthenticationManager();
68  }
69
70  @Bean
71  public PasswordEncoder passwordEncoder() {
72      return new BCryptPasswordEncoder();
73  }
74 }
```

La configuración de Spring Security define los recursos públicos y protegidos de la aplicación, además de establecer el uso de autenticación basada en JWT y una política de sesiones Stateless.

Autenticación mediante JWT

La autenticación se implementó mediante JSON Web Token (JWT). Cuando un usuario inicia sesión correctamente, el sistema genera un token firmado digitalmente que identifica al usuario autenticado.

Este token debe enviarse en todas las solicitudes dirigidas a recursos protegidos, permitiendo validar la identidad del usuario sin necesidad de mantener sesiones en el servidor.

Captura de Postman realizando el login y mostrando el token JWT.

La imagen muestra la autenticación de un usuario mediante el endpoint `/auth/login`, obteniendo un token JWT válido que posteriormente es utilizado para acceder a los recursos protegidos.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8081/auth/login
- Body:**

```

1 {
2   "username": "admin",
3   "password": "admin123"
4 }

```
- Response:**
 - Status:** 200 OK
 - Time:** 215 ms
 - Size:** 564 B
 - Content Type:** application/json
 - Body:**

```

1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cGE6ICJpYyJhdHkiOiJST09SVNUUkFET1IifV9sIm1hdCI6MTc0MdkzMjM2NywiZXhwIjoxNzg0MDA4MzY3fQ.DWCAAvIN4s86BxBeamCcG4i_H_wifnd-M567TKnL8oAt7S0QzCF4gPyb7YGZ4tP"
3 }

```

Bearer Token authorization.' The bottom panel shows the 'Body' tab with a JSON response: {"id": 1, "nombre": "Coca Cola 2L", "precio": 2500.0, "stock": 100}."/>

GET http://localhost:8081/api/productos

Docs Params **Authorization** Headers 8 Body Scripts Settings

Auth Type

Bearer ...

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token

200 OK • 44 ms • 37.73 KB

Body Cookies Headers 11 Test Results

{ } JSON Preview Visualize

```

1  [
2    {
3      "id": 1,
4      "nombre": "Coca Cola 2L",
5      "precio": 2500.0,
6      "stock": 100,

```

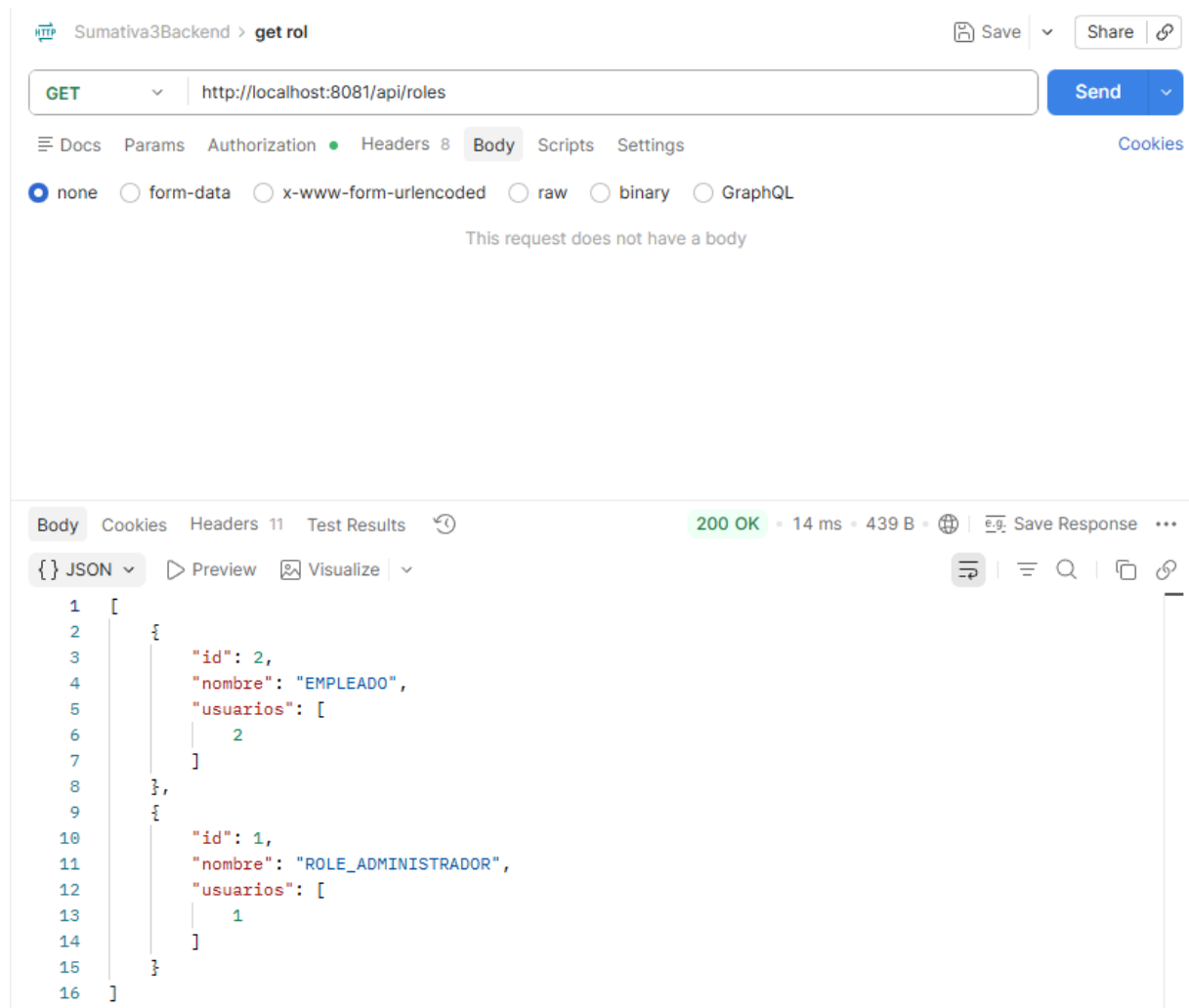
Autorización basada en roles

Para controlar el acceso a las distintas funcionalidades del sistema se implementaron tres roles:

- ROLE_ADMINISTRADOR
- ROLE_EMPLEADO
- ROLE_CLIENTE

Cada rol posee permisos específicos definidos dentro de la configuración de seguridad, permitiendo restringir el acceso a determinados endpoints según las responsabilidades de cada tipo de usuario.

Captura de ROLE_ADMINISTRADOR accediendo correctamente a un endpoint protegido.

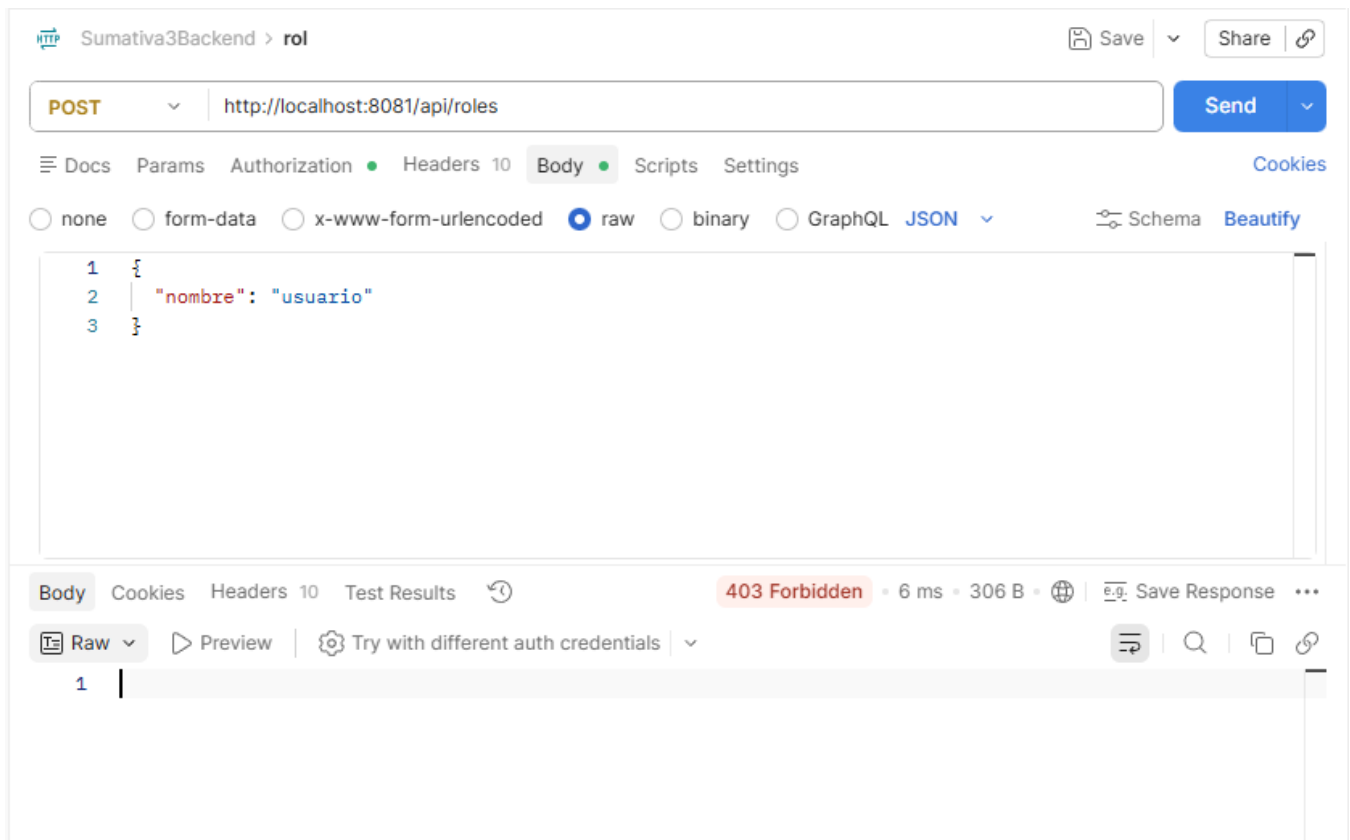


The screenshot shows a REST client interface with the following details:

- Request:** Method `GET`, URL `http://localhost:8081/api/roles`.
- Response:** Status `200 OK`, Time `14 ms`, Size `439 B`.
- Body (JSON):**

```
[
  {
    "id": 2,
    "nombre": "EMPLEADO",
    "usuarios": [
      2
    ]
  },
  {
    "id": 1,
    "nombre": "ROLE_ADMINISTRADOR",
    "usuarios": [
      1
    ]
  }
]
```

Captura de Empleado o cliente obteniendo respuesta **403 Forbidden** al intentar acceder a un recurso restringido.



Las pruebas realizadas permitieron comprobar que las reglas de autorización configuradas funcionan correctamente, permitiendo el acceso únicamente a los usuarios que poseen los permisos correspondientes.

Filtro de autenticación JWT

La validación del token se realiza mediante `JwtAuthenticationFilter`, encargado de interceptar cada solicitud entrante y verificar la existencia y validez del token JWT.

Si el token es válido, el usuario es autenticado y la solicitud continúa su procesamiento. En caso contrario, el acceso al recurso es rechazado automáticamente.

Captura de clase `JwtAuthenticationFilter.java`

El filtro JWT intercepta las solicitudes protegidas y valida el token antes de permitir el acceso a los controladores de la aplicación.

```
17
18 @Component
19 public class JwtAuthenticationFilter extends OncePerRequestFilter {
20
21     private final JwtUtil jwtUtil;
22     private final CustomUserDetailsService userDetailsService;
23
24     public JwtAuthenticationFilter(JwtUtil jwtUtil, CustomUserDetailsService userDetailsService) {
25         this.jwtUtil = jwtUtil;
26         this.userDetailsService = userDetailsService;
27     }
28
29     @Override
30     protected void doFilterInternal(HttpServletRequest request,
31                                     HttpServletResponse response,
32                                     FilterChain filterChain)
33         throws ServletException, IOException {
34
35         String header = request.getHeader(name: "Authorization");
36
37         if (header == null || !header.startsWith(prefix: "Bearer ")) {
38             filterChain.doFilter(request, response);
39             return;
40         }
41
42         String token = header.substring(beginIndex: 7);
43         String username = jwtUtil.extractUsername(token);
44
45         if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
46             UserDetails userDetails = userDetailsService.loadUserByUsername(username);
47
48             if (jwtUtil.validateToken(token, userDetails)) {
49                 UsernamePasswordAuthenticationToken authToken =
50                     new UsernamePasswordAuthenticationToken(
51                         userDetails,
52                         credentials: null,
53                         userDetails.getAuthorities()
54                     );
55
56                 authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
57                 SecurityContextHolder.getContext().setAuthentication(authToken);
58             }
59
60             filterChain.doFilter(request, response);
61         }
62     }
63 }
```

Protección de endpoints

Finalmente, se verificó el funcionamiento de la configuración de seguridad realizando pruebas sobre los principales endpoints del sistema.

Las pruebas permitieron comprobar que:

- Los endpoints públicos pueden ser accedidos sin autenticación.
- Los endpoints protegidos requieren un token JWT válido.
- Las restricciones por rol se aplican correctamente.

Pruebas unitarias

Con el propósito de verificar el correcto funcionamiento de la aplicación, se implementó un conjunto de pruebas unitarias utilizando JUnit 5 y Mockito. Estas pruebas permitieron validar la lógica de negocio de los principales servicios del sistema de forma aislada, simulando las dependencias mediante objetos mock y evitando la interacción directa con la base de datos. Para medir el nivel de validación alcanzado, se utilizó JaCoCo, herramienta que permitió generar reportes de cobertura y comprobar que las funcionalidades principales fueron evaluadas mediante pruebas automatizadas.

Configuración del entorno de pruebas

El proyecto fue configurado utilizando Maven, incorporando las dependencias necesarias para ejecutar pruebas unitarias y generar reportes de cobertura.

Entre las principales herramientas utilizadas se encuentran:

- JUnit 5.
- Mockito.
- Spring Boot Test.
- Spring Security Test.
- JaCoCo.

Captura de pom.xml de Junit5 y mockito que viene incluido dentro

```
62     <dependency>
63         <groupId>org.springframework.boot</groupId>
64         <artifactId>spring-boot-starter-test</artifactId>
65         <scope>test</scope>
66     </dependency>
```

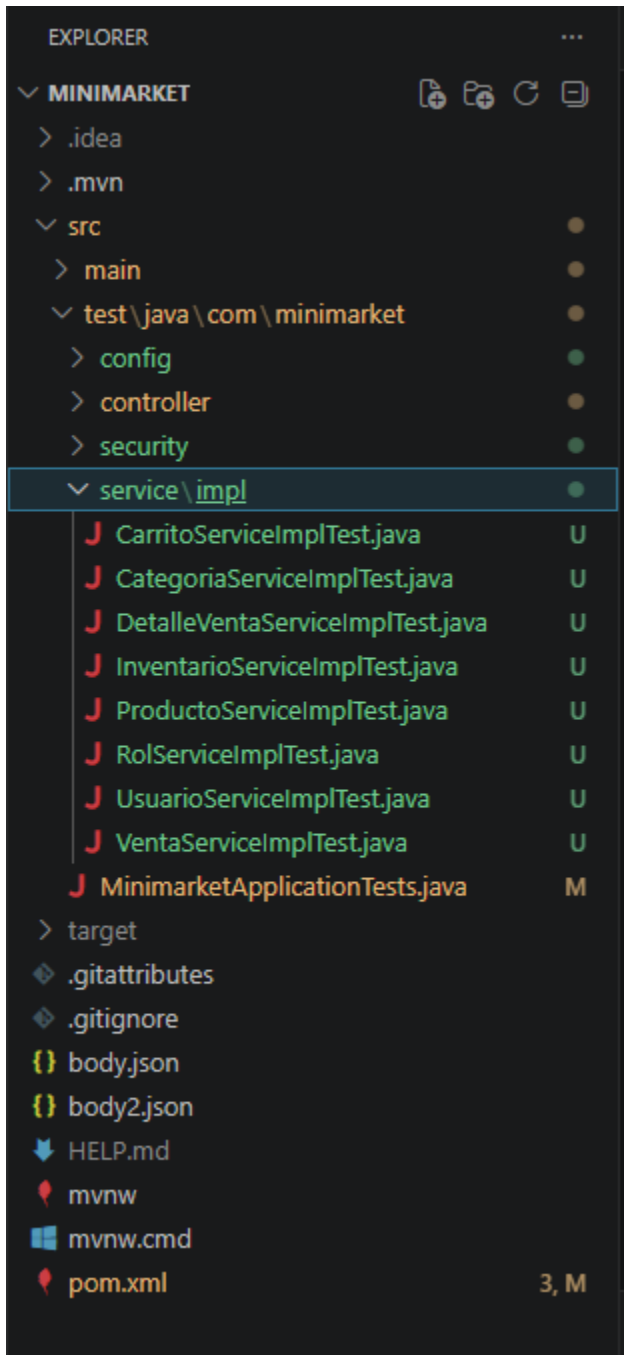
Captura de pom.xml de JaCoCo (plugin de Maven)

```
109     <plugin>
110         <groupId>org.jacoco</groupId>
111         <artifactId>jacoco-maven-plugin</artifactId>
112         <version>0.8.12</version>
113         <executions>
114             <execution>
115                 <goals>
116                     <goal>prepare-agent</goal>
117                 </goals>
118             </execution>
119             <execution>
120                 <id>report</id>
121                 <phase>test</phase>
122                 <goals>
123                     <goal>report</goal>
124                 </goals>
125             </execution>
126         </executions>
127     </plugin>
```

Estas dependencias permitieron implementar pruebas unitarias, simular dependencias mediante objetos mock, validar datos de entrada y generar reportes de cobertura del código.

Organización de las pruebas

Las clases de prueba fueron organizadas dentro del directorio `src/test/java`, siguiendo la misma estructura de paquetes utilizada por el código fuente del proyecto. Cada clase fue nombrada utilizando el sufijo `Test`, facilitando su identificación y ejecución automática por parte de Maven.



Implementación de las pruebas

Se desarrollaron pruebas unitarias para los servicios correspondientes a las entidades Producto, Inventario, Venta, Usuario y Carrito. Para ello se utilizó Mockito, simulando el comportamiento de los repositorios mediante anotaciones como `@Mock` e `@InjectMocks`, permitiendo validar únicamente la lógica de negocio implementada en la capa de servicios.

Se implementaron pruebas para las siguientes entidades:

Servicio	Validaciones realizadas
Producto	Consulta, creación, actualización y eliminación
Inventario	Registro y consulta de movimientos
Usuario	Búsqueda, almacenamiento y autenticación
Venta	Registro y consulta de ventas
Carrito	Administración de productos del carrito
JWT	Generación y validación de tokens

Ejemplo:

```
16
17 @ExtendWith(MockitoExtension.class)
18 class ProductoServiceImplTest {
19
20     @Mock
21     private ProductoRepository productoRepository;
22
23     @InjectMocks
24     private ProductoServiceImpl productoService;
25
26     @Test
27     void findAll_debeRetornarListaDeProductos() {
28         Producto producto = new Producto();
29         producto.setId(id: 1L);
30         producto.setNombre(nombre: "Pan");
31
32         when(productoRepository.findAll()).thenReturn(List.of(producto));
33
34         List<Producto> resultado = productoService.findAll();
35
36         assertEquals(1, resultado.size());
37         assertEquals("Pan", resultado.get(index: 0).getNombre());
38     }
39
```

Ejecución de las pruebas

Una vez implementadas, las pruebas fueron ejecutadas utilizando Maven mediante el comando:

“mvn clean test”

La ejecución permitió validar el funcionamiento de los componentes desarrollados y generar automáticamente el reporte de cobertura mediante JaCoCo.

Captura de Terminal ejecutando mvn clean test.

```
PS C:\Users\Sunnys\Downloads\minimarketnuevo\minimarket> mvn clean test
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.697 s -- in com.minimarket.MinimarketApplicationTests
[INFO] Running com.minimarket.service.impl.ProductoServiceImplTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.379 s -- in com.minimarket.service.impl.ProductoServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\Sunnys\Downloads\minimarketnuevo\minimarket\target\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 36 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 12.247 s
[INFO] Finished at: 2026-06-24T17:58:00-04:00
[INFO]
PS C:\Users\Sunnys\Downloads\minimarketnuevo\minimarket>
```

```
PS C:\Users\Sunnys\Downloads\minimarketnuevo\minimarket> mvn clean test
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.403 s -- in com.minimarket.service.impl.InventarioServiceImplTest
[INFO] Running com.minimarket.service.impl.ProductoServiceImplTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.096 s -- in com.minimarket.service.impl.ProductoServiceImplTest
[INFO] Running com.minimarket.service.impl.UsuarioServiceImplTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.085 s -- in com.minimarket.service.impl.UsuarioServiceImplTest
[INFO] Running com.minimarket.service.impl.VentaServiceImplTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.072 s -- in com.minimarket.service.impl.VentaServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\Sunnys\Downloads\minimarketnuevo\minimarket\target\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 36 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 12.614 s
[INFO] Finished at: 2026-06-24T18:08:09-04:00
[INFO]
PS C:\Users\Sunnys\Downloads\minimarketnuevo\minimarket>
```

(Las imágenes muestran la evolución de la ejecución de las pruebas unitarias, desde las primeras validaciones realizadas durante el desarrollo hasta alcanzar un total de 67 pruebas ejecutadas correctamente)

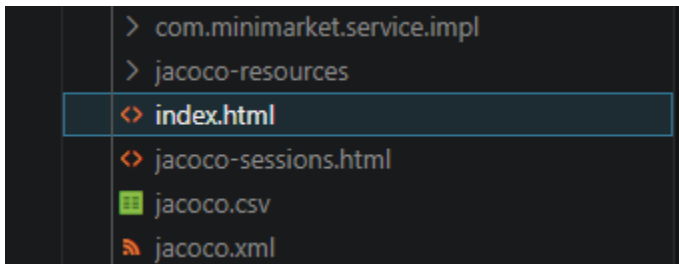
Cobertura de código

Para evaluar el nivel de validación alcanzado se utilizó JaCoCo, generando un reporte de cobertura que permitió identificar las clases evaluadas mediante pruebas automatizadas.

El análisis de cobertura permitió comprobar que los principales componentes del backend fueron considerados dentro del proceso de validación, aumentando la confiabilidad del sistema y reduciendo el riesgo de errores durante futuras modificaciones.

Captura de reporte HTML de JaCoCo

El reporte generado por JaCoCo muestra la cobertura obtenida durante la ejecución de las pruebas unitarias, permitiendo verificar que los principales componentes del sistema fueron evaluados correctamente.



Reporte de cobertura de JaCoCo

 minimarket

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 com.minimarket.controller		6 %		0 %	68	78	131	146	48	58	2	12
 com.minimarket.entity		36 %		n/a	47	74	68	107	47	74	0	8
 com.minimarket.service.impl		53 %		n/a	18	42	22	48	18	42	0	8
 com.minimarket.security.util		4 %		0 %	8	9	17	18	6	7	0	1
 com.minimarket.security.filter		12 %		0 %	6	7	15	19	1	2	0	1
 com.minimarket.security.model		0 %		n/a	14	14	19	19	14	14	2	2
 com.minimarket.security.service		13 %		n/a	2	3	3	4	2	3	0	1
 com.minimarket		37 %		n/a	1	2	2	3	1	2	0	1
 com.minimarket.security.config		100 %		n/a	0	9	0	25	0	9	0	1
 com.minimarket.config		100 %		50 %	1	4	0	17	0	3	0	1
Total	991 of 1.540	35 %	55 of 56	1 %	165	242	277	406	137	214	4	36

Resultados obtenidos

Una vez implementadas las pruebas unitarias, se ejecutó el comando `mvn clean test` para verificar el funcionamiento del proyecto. Durante esta primera ejecución, todas las pruebas fueron completadas correctamente, sin presentar errores ni fallos, permitiendo además generar el primer reporte de cobertura mediante JaCoCo.

El análisis inicial mostró una cobertura aproximada del 35 %, concentrándose principalmente en la capa de servicios. Este resultado evidenció que todavía existían componentes del sistema que no estaban siendo evaluados por las pruebas unitarias, especialmente controladores, clases de configuración y algunos componentes de seguridad.

La imagen muestra el primer reporte generado por JaCoCo después de ejecutar las pruebas unitarias. En esta etapa se observó una cobertura aproximada del 35 %, permitiendo identificar las clases que aún requerían la incorporación de nuevos casos de prueba para aumentar el nivel de validación del sistema.

Las pruebas ejecutadas permitieron verificar el correcto funcionamiento de la lógica de negocio implementada en los servicios del sistema, confirmando que las operaciones principales responden de acuerdo con los requerimientos definidos.

Además, la utilización de pruebas automatizadas facilitó la detección temprana de posibles errores y contribuyó a fortalecer la estabilidad y mantenibilidad del backend.

 minimarket

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.minimarket.controller		6 %		0 %	68	78	131	146	48	58	2	12
com.minimarket.entity		36 %		n/a	47	74	68	107	47	74	0	8
com.minimarket.service.impl		53 %		n/a	18	42	22	48	18	42	0	8
com.minimarket.security.util		4 %		0 %	8	9	17	18	6	7	0	1
com.minimarket.security.filter		12 %		0 %	6	7	15	19	1	2	0	1
com.minimarket.security.model		0 %		n/a	14	14	19	19	14	14	2	2
com.minimarket.security.service		13 %		n/a	2	3	3	4	2	3	0	1
com.minimarket		37 %		n/a	1	2	2	3	1	2	0	1
com.minimarket.security.config		100 %		n/a	0	9	0	25	0	9	0	1
com.minimarket.config		100 %		50 %	1	4	0	17	0	3	0	1
Total	991 of 1.540	35 %	55 of 56	1 %	165	242	277	406	137	214	4	36

A partir del análisis del primer reporte de cobertura, se desarrollaron nuevas pruebas unitarias y se fortalecieron las ya existentes para aumentar la cantidad de clases y métodos evaluados. Durante este proceso se incorporaron pruebas para las entidades Producto, Inventario, Venta, Usuario y Carrito, además de realizar mejoras en la configuración de seguridad, la autenticación mediante JWT, la validación de datos de entrada y la unificación de los roles utilizados por Spring Security.

Estas modificaciones permitieron incrementar significativamente la cobertura del proyecto y mejorar la confiabilidad general del backend.

Además del aumento en la cobertura del código, se verificó el correcto funcionamiento de los mecanismos de autenticación y autorización implementados mediante Spring Security. Durante las pruebas se comprobó que los endpoints públicos, como /auth/login, /public/ y /h2-console/, permanecieran accesibles sin autenticación, mientras que los endpoints correspondientes a la administración de usuarios, roles, categorías, productos, inventario, ventas y carrito de compras solo pudieran ser utilizados por los roles autorizados. De esta forma fue posible confirmar que las reglas de acceso definidas en la configuración de seguridad se aplicaban correctamente según el perfil de cada usuario.

También se evaluaron distintos escenarios de acceso no autorizado, verificando que el sistema rechazara correctamente las solicitudes realizadas sin token JWT, con tokens inválidos o expirados, o por usuarios que intentaban acceder a recursos para los cuales no contaban con permisos suficientes. Por ejemplo, un usuario con rol CLIENTE no puede acceder a los endpoints destinados a la administración de usuarios, roles o categorías, obteniendo una respuesta de acceso denegado conforme a la configuración establecida.

A partir de los resultados obtenidos se realizaron ajustes orientados a fortalecer la seguridad del proyecto. Entre ellos destacan la centralización de las reglas de autorización dentro de SecurityConfig, la incorporación del filtro JwtAuthenticationFilter para validar los tokens antes de procesar cada solicitud, la configuración de sesiones tipo STATELESS, la deshabilitación de CSRF al tratarse de una API REST y la unificación de los roles utilizando la convención ROLE_ADMINISTRADOR, ROLE_EMPLEADO y ROLE_CLIENTE. Asimismo, durante el análisis se identificó como recomendación fortalecer la clave secreta utilizada para la firma de

los tokens JWT, ya que una longitud mayor proporciona un nivel de seguridad más adecuado para ambientes de producción.

Luego de incorporar las mejoras identificadas durante el análisis inicial, se ejecutó nuevamente el conjunto completo de pruebas utilizando el comando `mvn clean test`.

Como resultado, las 67 pruebas unitarias fueron ejecutadas correctamente, sin errores ni fallos, obteniéndose además un 100 % de cobertura según el reporte generado por JaCoCo.

Este resultado demuestra que las principales funcionalidades implementadas fueron evaluadas mediante pruebas automatizadas y que las mejoras incorporadas permitieron aumentar considerablemente la cobertura del código, fortaleciendo la calidad y mantenibilidad del proyecto.

Reporte final de cobertura generado por JaCoCo

La imagen presenta el reporte final generado por JaCoCo luego de incorporar nuevas pruebas unitarias y realizar las mejoras derivadas del análisis inicial. Se observa una cobertura del 100%, lo que demuestra que todas las clases y métodos considerados en el proyecto fueron evaluados satisfactoriamente mediante pruebas automatizadas.

minimarket

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.minimarket.controller		100 %		100 %	0	78	0	190	0	57	0	12
com.minimarket.hateoas		100 %		100 %	0	24	0	90	0	16	0	4
com.minimarket.security.config		100 %		n/a	0	9	0	30	0	9	0	1
com.minimarket.entity		100 %		n/a	0	74	0	107	0	74	0	8
com.minimarket.service.impl		100 %		n/a	0	42	0	48	0	42	0	8
com.minimarket.config		100 %		100 %	0	6	0	28	0	5	0	2
com.minimarket.exception		100 %		n/a	0	10	0	19	0	10	0	3
com.minimarket.security.filter		100 %		100 %	0	7	0	22	0	2	0	1
com.minimarket.security.util		100 %		n/a	0	7	0	23	0	7	0	1
com.minimarket.security.model		100 %		n/a	0	14	0	19	0	14	0	2
com.minimarket.security.service		100 %		n/a	0	3	0	4	0	3	0	1
com.minimarket		100 %		n/a	0	2	0	3	0	2	0	1
Total	0 of 2.378	100 %	0 of 70	100 %	0	276	0	583	0	241	0	44

Captura de ejemplo:

La imagen muestra la ejecución del comando `mvn clean test`, obteniendo una ejecución satisfactoria de 67 pruebas unitarias, sin errores ni fallos. Esto confirma la estabilidad del proyecto y el correcto funcionamiento de las pruebas implementadas.

```
PS C:\Users\Sunnys\Desktop\minimarketnuevo\minimarket> .\mvnw.cmd test
[INFO] Results:
[INFO]
[INFO] Tests run: 67, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\Sunnys\Desktop\minimarketnuevo\minimarket\target\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 44 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 14.739 s
[INFO] Finished at: 2026-07-18T00:05:33-04:00
[INFO] -----
❖ PS C:\Users\Sunnys\Desktop\minimarketnuevo\minimarket> 
```

Tras incorporar las mejoras relacionadas con OpenAPI, HATEOAS y el manejo global de excepciones, se actualizaron las pruebas del proyecto para validar las nuevas respuestas HATEOAS, el código HTTP 201 Created, el encabezado Location y el formato uniforme de errores. Finalmente, se ejecutaron 67 pruebas unitarias, obteniendo 0 fallos y 0 errores.

Documentación de la API con OpenAPI y HATEOAS

Implementación de HATEOAS

Con el propósito de mejorar la navegación de la API y cumplir con las buenas prácticas de las arquitecturas REST, se incorporó Spring HATEOAS en los principales recursos del sistema Minimarket Plus. Para ello se agregó la dependencia `spring-boot-starter-hateoas` y se modificaron los controladores para devolver respuestas enriquecidas mediante `EntityModel` y `CollectionModel`.

Cada recurso incluye enlaces (`_links`) que permiten acceder directamente al propio recurso (self) y a la colección correspondiente. De esta forma, el cliente puede descubrir dinámicamente los recursos relacionados sin necesidad de conocer previamente todas las rutas disponibles de la API.

Para mejorar la mantenibilidad del proyecto, la generación de enlaces HATEOAS fue centralizada mediante `Model Assemblers`, evitando que cada controlador construya manualmente sus enlaces. Se implementaron las clases `ProductoModelAssembler`, `CarritoModelAssembler`, `InventarioModelAssembler` y `UsuarioModelAssembler`, encargadas de generar de forma consistente los enlaces asociados a cada recurso.

Las pruebas realizadas mediante Postman confirmaron que los endpoints retornan correctamente tanto los datos solicitados como los enlaces generados automáticamente, evidenciando una implementación funcional de HATEOAS sobre los recursos principales del sistema.

Implementación de HATEOAS mediante Model Assemblers

Con el objetivo de mejorar la organización y mantenibilidad del proyecto, la implementación de Spring HATEOAS fue refactorizada mediante el uso de Model Assemblers. Para ello se crearon las clases `ProductoModelAssembler`, `CarritoModelAssembler`, `InventarioModelAssembler` y `UsuarioModelAssembler`, responsables de construir los enlaces hipermedia asociados a cada recurso.

Esta implementación permite que los controladores deleguen la construcción de las respuestas HATEOAS a componentes especializados, evitando la duplicidad de código y manteniendo una estructura más limpia y reutilizable. De esta forma, los controladores se enfocan únicamente en atender las solicitudes y coordinar la lógica de negocio, mientras que los assemblers generan automáticamente los enlaces correspondientes.

Cada assembler incorpora enlaces dinámicos como `self`, actualizar, eliminar y otros recursos relacionados, además de enlaces hacia la colección principal y entidades asociadas cuando la información está disponible. Esto mejora la navegación dentro de la API y mantiene una implementación consistente entre todos los recursos.

Captura de implementación de ProductoModelAssembler

La siguiente captura muestra un ejemplo de la implementación del ProductoModelAssembler, encargado de construir la representación HATEOAS del recurso Producto y de agregar automáticamente los enlaces disponibles para cada respuesta.

```
17
18 @Component
19 public class ProductoModelAssembler {
20
21     public EntityModel<Producto> toModel(Producto producto) {
22         List<Link> links = new ArrayList<>(List.of(
23             linkTo(methodOn(controller: ProductoController.class)
24                 .obtenerProductoPorId(producto.getId())).withSelfRel(),
25             linkTo(methodOn(controller: ProductoController.class)
26                 .listarProductos()).withRel(rel: "productos"),
27             linkTo(methodOn(controller: ProductoController.class)
28                 .actualizarProducto(producto.getId(), producto)).withRel(rel: "actualizar"),
29             linkTo(methodOn(controller: ProductoController.class)
30                 .eliminarProducto(producto.getId())).withRel(rel: "eliminar")
31         ));
32
33         if (producto.getCategoria() != null && producto.getCategoria().getId() != null) {
34             links.add(linkTo(methodOn(controller: CategoriaController.class)
35                 .obtenerCategoriaPorId(producto.getCategoria().getId())).withRel(rel: "categoria"));
36         }
37
38         return EntityModel.of(producto, links);
39     }
40
41     public CollectionModel<EntityModel<Producto>> toCollectionModel(List<Producto> productos) {
42         List<EntityModel<Producto>> productoModels = productos.stream()
43             .map(this::toModel)
44             .toList();
45
46         return CollectionModel.of(productoModels,
47             linkTo(methodOn(controller: ProductoController.class)
48                 .listarProductos()).withSelfRel()
49         );
50     }
51
52     public URI toSelfUri(Producto producto) {
53         return linkTo(methodOn(controller: ProductoController.class)
54             .obtenerProductoPorId(producto.getId())).toUri();
55     }
56 }
57
```


Documentación de la API

Con el objetivo de facilitar el consumo de los servicios REST y mejorar la comprensión de la API, se implementó documentación automática mediante OpenAPI Specification (Swagger) y navegación entre recursos utilizando Spring HATEOAS.

La documentación generada permite visualizar los endpoints disponibles, los parámetros requeridos, los modelos de datos y las respuestas esperadas, facilitando las pruebas y la integración con aplicaciones externas.

Dependencias Maven incorporadas para OpenAPI, HATEOAS y seguridad en el archivo pom.xml.

Spring Security

Esta dependencia permitió implementar la seguridad del backend mediante Spring Security. Fue utilizada para proteger los endpoints de la API, controlar el acceso según los roles de los usuarios y complementar el proceso de autenticación basado en JWT.

```
36      </dependency>
37      <dependency>
38          <groupId>org.springframework.boot</groupId>
39          <artifactId>spring-boot-starter-security</artifactId>
40      </dependency>
```

Spring Validation

Esta dependencia incorpora el soporte para la validación de datos recibidos por la API mediante anotaciones como @NotNull, @NotBlank y otras restricciones, contribuyendo a mantener la integridad de la información antes de almacenarla.

```
89      <dependency>
90          <groupId>org.springframework.boot</groupId>
91          <artifactId>spring-boot-starter-validation</artifactId>
92      </dependency>
```

OpenApi (Swagger)

La dependencia `springdoc-openapi-starter-webmvc-ui` permitió generar automáticamente la documentación de la API utilizando el estándar OpenAPI y visualizarla mediante Swagger UI. Además de documentar los endpoints, se incorporaron ejemplos de solicitudes mediante `@ExampleObject` y esquemas detallados utilizando `@Schema`, facilitando la comprensión de los modelos utilizados por la API.

Cada controlador fue documentado utilizando anotaciones como:

- `@Tag`
- `@Operation`
- `@Parameter`
- `@ApiResponses`
- `@ApiResponse`
- `@Schema`
- `@ExampleObject`

Gracias a ello, la documentación permanece sincronizada con la implementación del proyecto.

```
93 |         <dependency>
94 |             <groupId>org.springdoc</groupId>
95 |             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
96 |             <version>2.8.9</version>
97 |         </dependency>
```

Spring HATEOAS

Esta dependencia permitió incorporar HATEOAS en las respuestas de la API mediante `EntityModel` y `CollectionModel`. La construcción de enlaces utilizando `linkTo()` y `methodOn()` fue centralizada en las clases `Model Assembler`, evitando duplicidad de código en los controladores y manteniendo una navegación consistente entre los recursos relacionados.

```
98 |         <dependency>
99 |             <groupId>org.springframework.boot</groupId>
100 |             <artifactId>spring-boot-starter-hateoas</artifactId>
101 |         </dependency>
```

Captura de Swagger mostrando todos los controladores.

Esta captura muestra la documentación general generada automáticamente por Swagger UI para el proyecto Minimarket Plus, donde se observan los controladores implementados y los endpoints disponibles.

The screenshot displays the Swagger UI interface for the Minimarket Plus API. At the top, the Swagger logo and version (3.0.0) are visible, along with the API name 'Minimarket Plus API' and version '1.0' (OAS 3.1). The API is described as 'API REST para la gestión de productos, inventario, ventas, usuarios y carrito del sistema Minimarket Plus.' Below this, there is a 'Servers' section with a dropdown menu showing 'http://localhost:8081 - Generated server url'. The main content area lists two groups of endpoints: 'Carrito' (Endpoints para la gestión del carrito de compras del sistema Minimarket Plus) and 'Roles' (Endpoints para la gestión de roles de usuario). Each group contains several endpoints with their respective HTTP methods and descriptions.

Method	Endpoint	Description
GET	/api/carrito/{id}	Obtener registro del carrito por ID
PUT	/api/carrito/{id}	Actualizar registro del carrito
DELETE	/api/carrito/{id}	Eliminar producto del carrito
GET	/api/carrito	Listar carrito
POST	/api/carrito	Agregar producto al carrito

Method	Endpoint	Description
GET	/api/roles	Listar roles
POST	/api/roles	Crear rol

Inventario Endpoints para la gestion de movimientos de inventario

GET	/api/inventario/{id}	Obtener movimiento de inventario por ID	⌵
PUT	/api/inventario/{id}	Actualizar movimiento de inventario	⌵
DELETE	/api/inventario/{id}	Eliminar movimiento de inventario	⌵
GET	/api/inventario	Listar movimientos de inventario	⌵
POST	/api/inventario	Registrar movimiento de inventario	⌵

Autenticacion Endpoint para autenticacion de usuarios y emision de tokens JWT

POST	/auth/login	Iniciar sesion	⌵
------	-------------	----------------	---

Publico Endpoints publicos de prueba

GET	/public/hola	Saludo publico	⌵
-----	--------------	----------------	---

Categorias Endpoints para la gestion de categorias de productos

GET	/api/categorias/{id}	Obtener categoria por ID	⌵
PUT	/api/categorias/{id}	Actualizar categoria	⌵
DELETE	/api/categorias/{id}	Eliminar categoria	⌵
GET	/api/categorias	Listar categorias	⌵
POST	/api/categorias	Crear categoria	⌵

Productos Endpoints para la gestión de productos del sistema Minimarket Plus

GET /api/productos/{id} Obtener producto por ID

PUT /api/productos/{id} Actualizar producto

DELETE /api/productos/{id} Eliminar producto

GET /api/productos Listar productos

POST /api/productos Crear producto

Ventas Endpoints para la gestión de ventas del sistema Minimarket Plus

GET /api/ventas Listar ventas

POST /api/ventas Crear venta

GET /api/ventas/{id} Obtener venta por ID

Usuarios Endpoints para la gestión de usuarios del sistema Minimarket Plus

GET /api/usuarios/{id} Obtener usuario por ID

PUT /api/usuarios/{id} Actualizar usuario

DELETE /api/usuarios/{id} Eliminar usuario

GET /api/usuarios Listar usuarios

POST /api/usuarios Crear usuario

Detalles de venta Endpoints para la gestión de detalles asociados a ventas

GET /api/detalle-ventas/{id} Obtener detalle de venta por ID

PUT /api/detalle-ventas/{id} Actualizar detalle de venta

DELETE /api/detalle-ventas/{id} Eliminar detalle de venta

GET /api/detalle-ventas Listar detalles de venta

POST /api/detalle-ventas Crear detalle de venta

Documentación del endpoint POST/api/productos con parámetros, respuestas y modelo de datos.

En esta evidencia se presenta la documentación del endpoint encargado de registrar un nuevo producto. Swagger UI muestra la descripción de la operación, el cuerpo de solicitud con un ejemplo JSON y las posibles respuestas HTTP. Entre ellas se encuentra el código 201 Created, utilizado cuando el producto es registrado correctamente, además de las respuestas 400 Bad Request y 500 Internal Server Error.

The image shows the Swagger UI for the endpoint `POST /api/productos` with the title "Crear producto". The description states: "Registra un nuevo producto en el sistema Minimarket Plus." There are no parameters. The request body is required and the media type is set to `application/json`. An example is provided under "Examples:" with a dropdown menu showing "Producto valido". Below this, the "Example Value" is displayed as a JSON object:

```
{  "nombre": "Pan integral",  "precio": 1500,  "stock": 25,  "categoria": {    "id": 1  }}
```

 The "Example Description" is "Producto valido".

POST /api/productos Crear producto

Registra un nuevo producto en el sistema Minimarket Plus.

Parameters Try it out

No parameters

Request body required application/json

Datos del producto que se desea registrar en el sistema

Examples:

Producto valido

Example Value | Schema

```
{  "nombre": "Pan integral",  "precio": 1500,  "stock": 25,  "categoria": {    "id": 1  }}
```

Example Description

Producto valido

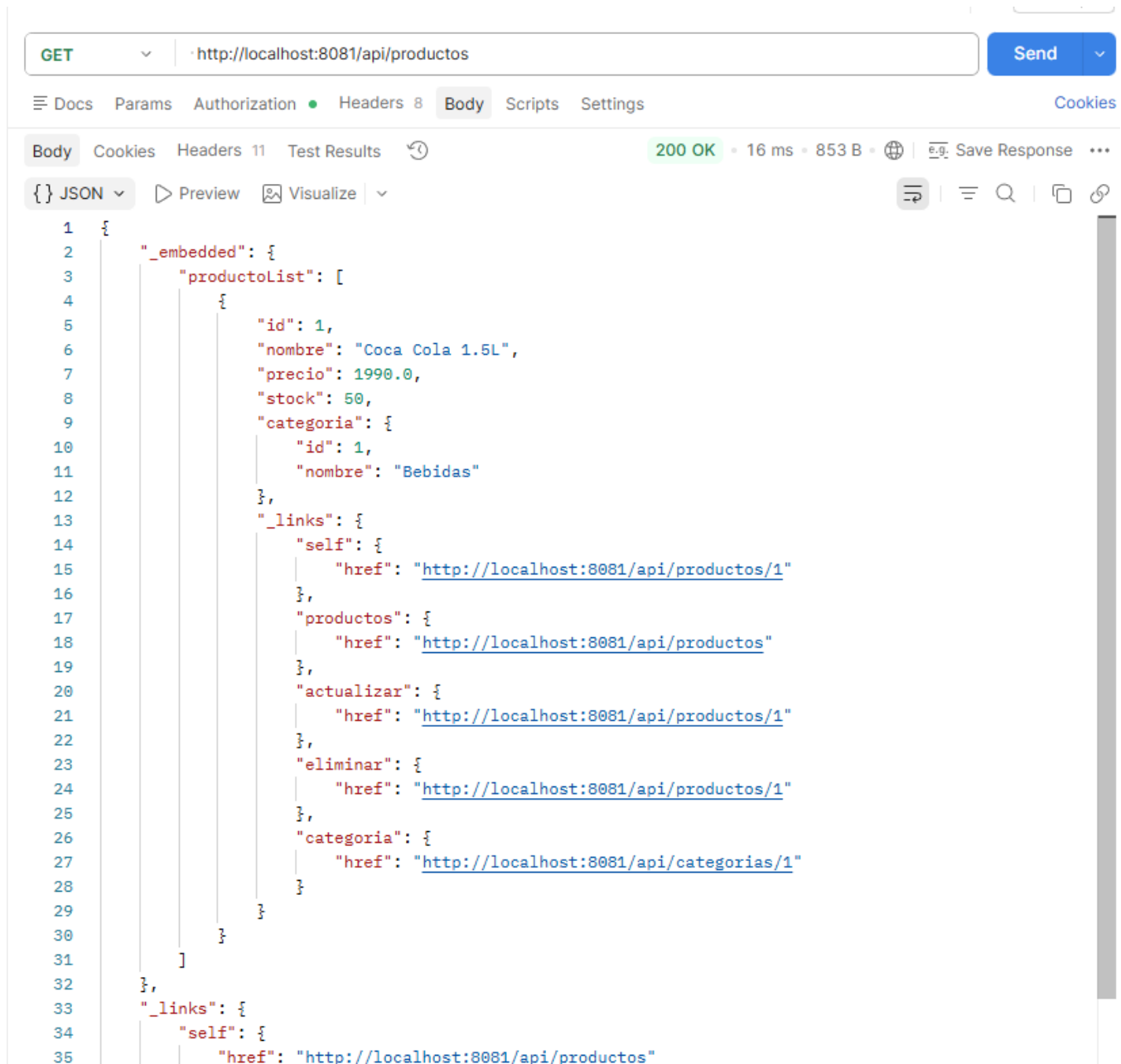
Responses

Code	Description	Links
201	Producto creado correctamente Media type */json Controls Accept header. Example Value Schema <pre>{ "id": 1, "nombre": "Coca Cola 1.5L", "precio": 1990, "stock": 50, "categoria": { "id": 1, "nombre": "Lacteos" } }</pre>	No links
400	Datos inválidos en la solicitud Media type */json Example Value Schema <pre>{ "id": 1, "nombre": "Coca Cola 1.5L", "precio": 1990, "stock": 50, "categoria": { "id": 1, "nombre": "Lacteos" } }</pre>	No links
500	Error interno del servidor Media type */json Example Value Schema <pre>{ "id": 1, "nombre": "Coca Cola 1.5L", "precio": 1990, "stock": 50, "categoria": { "id": 1, "nombre": "Lacteos" } }</pre>	No links

GET /api/productos

Respuesta del endpoint de productos con enlaces HATEOAS implementados.

La siguiente captura corresponde a la respuesta obtenida desde Postman para el endpoint de listado de productos. Se aprecia la estructura `_embedded` junto con los enlaces `_links` generados mediante Spring HATEOAS, permitiendo navegar hacia cada producto individual y hacia la colección completa de productos.



GET `http://localhost:8081/api/productos` Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body Cookies Headers 11 Test Results 200 OK • 16 ms • 853 B Save Response

JSON Preview Visualize

```
1 {
2   "_embedded": {
3     "productoList": [
4       {
5         "id": 1,
6         "nombre": "Coca Cola 1.5L",
7         "precio": 1990.0,
8         "stock": 50,
9         "categoria": {
10          "id": 1,
11          "nombre": "Bebidas"
12        },
13        "_links": {
14          "self": {
15            "href": "http://localhost:8081/api/productos/1"
16          },
17          "productos": {
18            "href": "http://localhost:8081/api/productos"
19          },
20          "actualizar": {
21            "href": "http://localhost:8081/api/productos/1"
22          },
23          "eliminar": {
24            "href": "http://localhost:8081/api/productos/1"
25          },
26          "categoria": {
27            "href": "http://localhost:8081/api/categorias/1"
28          }
29        }
30      }
31    ]
32  },
33  "_links": {
34    "self": {
35      "href": "http://localhost:8081/api/productos"
```


GET /api/carrito

Respuesta del endpoint de carrito utilizando navegación HATEOAS.

La siguiente evidencia muestra la respuesta obtenida desde el endpoint del carrito mediante Postman. Puede observarse la incorporación de enlaces HATEOAS (_links), los cuales permiten navegar hacia recursos relacionados de forma dinámica.

The screenshot shows the Postman interface with the following details:

- Method:** GET
- URL:** http://localhost:8081/api/carrito
- Status:** 200 OK
- Time:** 27 ms
- Size:** 1016 B
- Response Type:** JSON

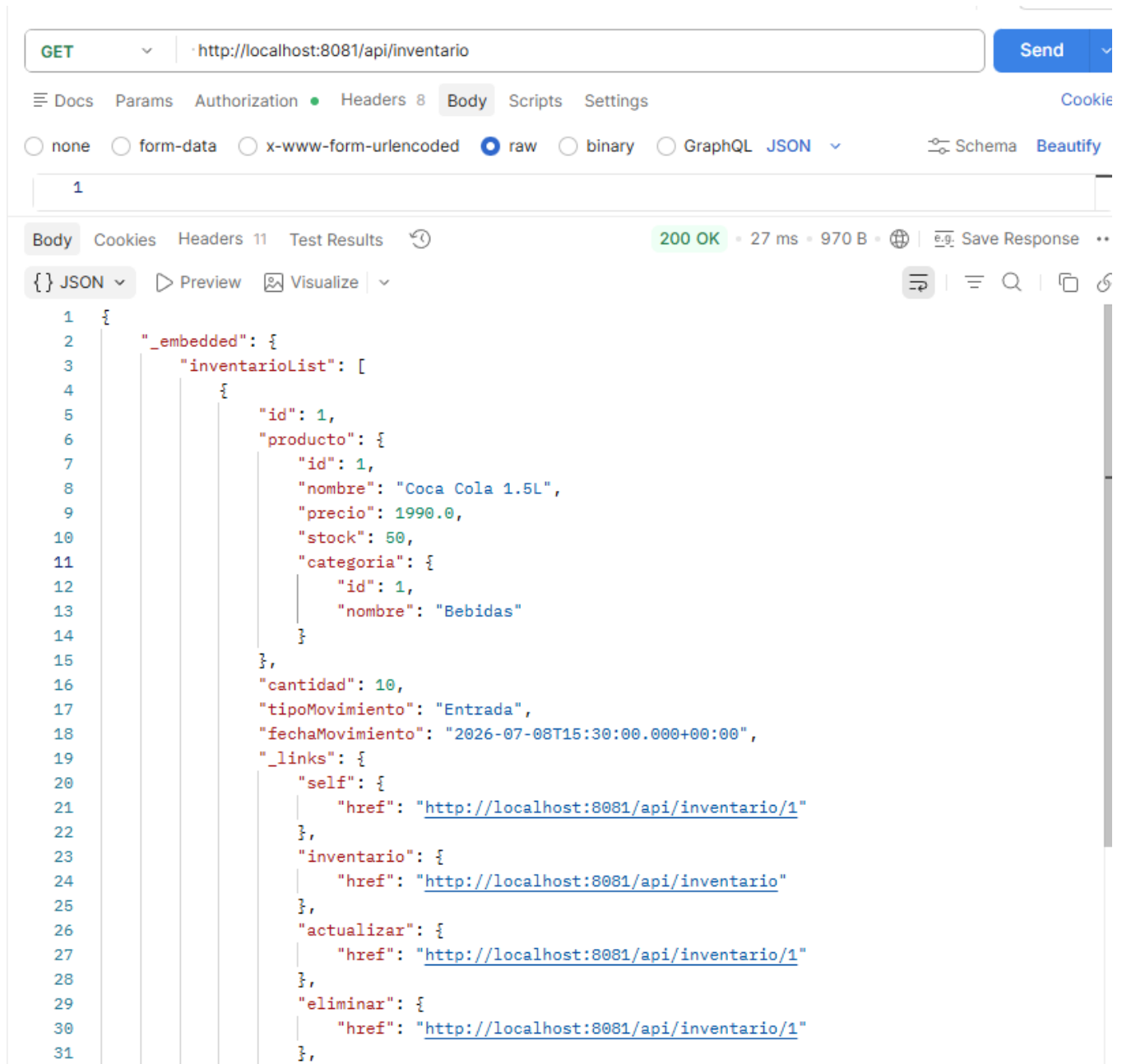
```
1  {
2    "_embedded": {
3      "carritoList": [
4        {
5          "id": 1,
6          "usuario": {
7            "id": 1,
8            "username": "admin",
9            "roles": [
10             {
11               "id": 1,
12               "nombre": "ROLE_ADMINISTRADOR"
13             }
14           ]
15         },
16         "producto": {
17           "id": 1,
18           "nombre": "Coca Cola 1.5L",
19           "precio": 1990.0,
20           "stock": 50,
21           "categoria": {
22             "id": 1,
23             "nombre": "Bebidas"
24           }
25         },
26         "cantidad": 2,
27         "_links": {
28           "self": {
29             "href": "http://localhost:8081/api/carrito/1"
30           },
31           "carrito": {
32             "href": "http://localhost:8081/api/carrito"
```

```
31     "carrito": {
32       "href": "http://localhost:8081/api/carrito"
33     },
34     "actualizar": {
35       "href": "http://localhost:8081/api/carrito/1"
36     },
37     "eliminar": {
38       "href": "http://localhost:8081/api/carrito/1"
39     },
40     "usuario": {
41       "href": "http://localhost:8081/api/usuarios/1"
42     },
43     "producto": {
44       "href": "http://localhost:8081/api/productos/1"
45     }
46   }
47 }
48 ]
49 },
50 "_links": {
51   "self": {
52     "href": "http://localhost:8081/api/carrito"
53   }
54 }
55 }
```

GET /api/inventario

Respuesta del endpoint de inventario utilizando enlaces HATEOAS.

Esta captura presenta la respuesta generada por el endpoint de inventario. Puede observarse que cada movimiento de inventario incorpora enlaces dinámicos HATEOAS, facilitando el acceso tanto al recurso específico como al listado general de movimientos registrados en el sistema.



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8081/api/inventario
- Status:** 200 OK
- Time:** 27 ms
- Size:** 970 B
- Format:** JSON
- Body:**

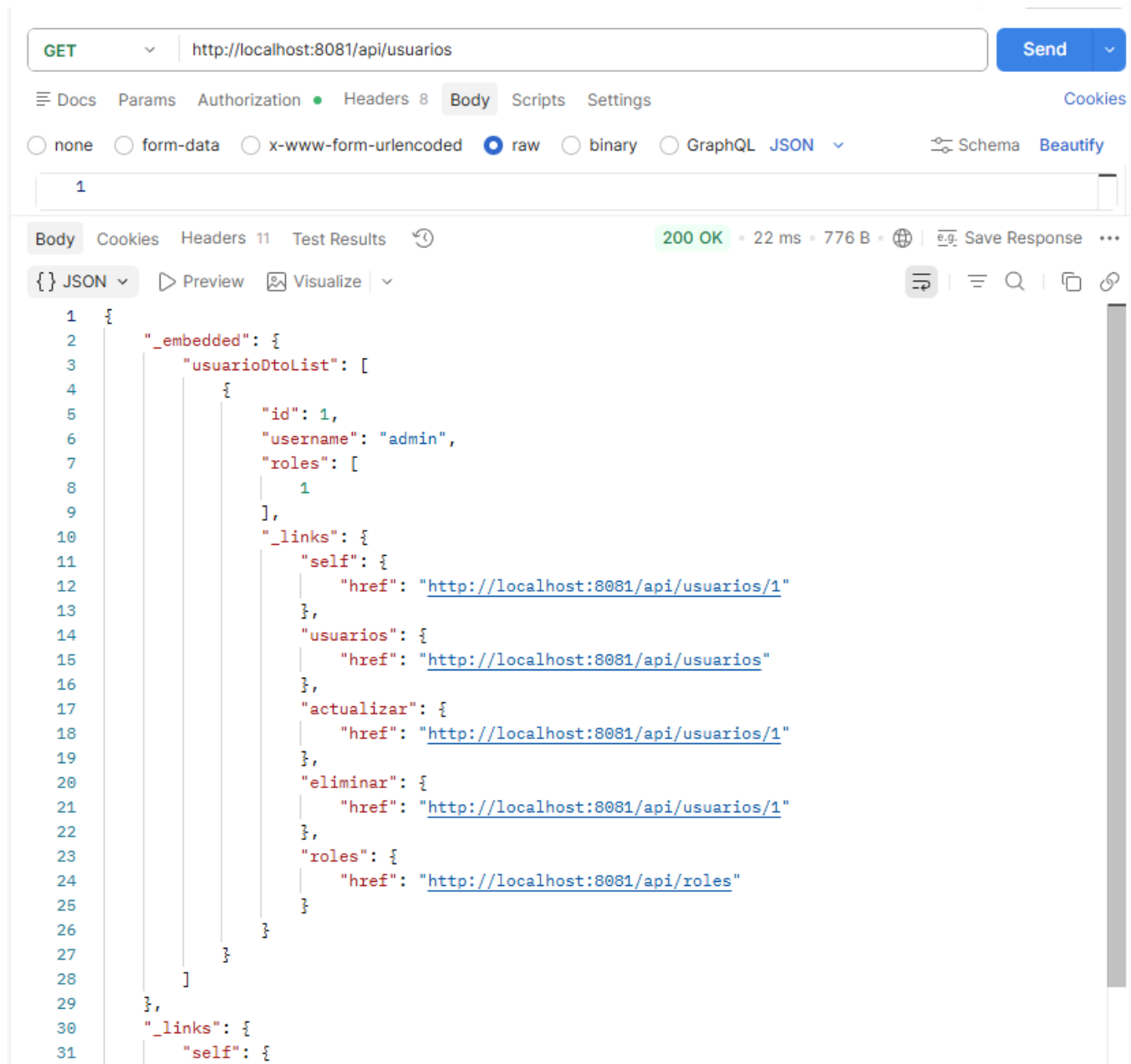
```
1 {
2   "_embedded": {
3     "inventarioList": [
4       {
5         "id": 1,
6         "producto": {
7           "id": 1,
8           "nombre": "Coca Cola 1.5L",
9           "precio": 1990.0,
10          "stock": 50,
11          "categoria": {
12            "id": 1,
13            "nombre": "Bebidas"
14          }
15        },
16        "cantidad": 10,
17        "tipoMovimiento": "Entrada",
18        "fechaMovimiento": "2026-07-08T15:30:00.000+00:00",
19        "_links": {
20          "self": {
21            "href": "http://localhost:8081/api/inventario/1"
22          },
23          "inventario": {
24            "href": "http://localhost:8081/api/inventario"
25          },
26          "actualizar": {
27            "href": "http://localhost:8081/api/inventario/1"
28          },
29          "eliminar": {
30            "href": "http://localhost:8081/api/inventario/1"
31          }
32        }
33      }
34    ]
35  }
36}
```

```
28     },
29     "eliminar": {
30       "href": "http://localhost:8081/api/inventario/1"
31     },
32     "producto": {
33       "href": "http://localhost:8081/api/productos/1"
34     }
35   }
36 }
37 ]
38 },
39 "_links": {
40   "self": {
41     "href": "http://localhost:8081/api/inventario"
42   }
43 }
44 }
```

GET /api/usuarios

Respuesta del endpoint de usuarios utilizando navegación HATEOAS.

La siguiente evidencia corresponde al endpoint de consulta de usuarios. La respuesta incorpora enlaces HATEOAS generados automáticamente, permitiendo acceder al usuario consultado y a la colección completa de usuarios, manteniendo una navegación consistente con el resto de la API.



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8081/api/usuarios
- Status:** 200 OK
- Time:** 22 ms
- Size:** 776 B
- Format:** JSON

The response body is a JSON object with the following structure:

```
1 {
2   "_embedded": {
3     "usuarioDtoList": [
4       {
5         "id": 1,
6         "username": "admin",
7         "roles": [
8           1
9         ],
10        "_links": {
11          "self": {
12            "href": "http://localhost:8081/api/usuarios/1"
13          },
14          "usuarios": {
15            "href": "http://localhost:8081/api/usuarios"
16          },
17          "actualizar": {
18            "href": "http://localhost:8081/api/usuarios/1"
19          },
20          "eliminar": {
21            "href": "http://localhost:8081/api/usuarios/1"
22          },
23          "roles": {
24            "href": "http://localhost:8081/api/roles"
25          }
26        }
27      }
28    ]
29  },
30  "_links": {
31    "self": {
```

```
31 |         "href": "http://localhost:8081/api/usuarios"
32 |     }
33 | }
34 | }
35 | }
```

La respuesta obtenida desde Postman evidencia el funcionamiento de HATEOAS, incorporando la sección `_links` con enlaces de navegación generados automáticamente. Además, la estructura `_embedded` contiene los recursos solicitados, permitiendo que las aplicaciones cliente puedan descubrir otros endpoints relacionados sin conocer previamente todas las rutas de la API.

Estandarización de respuestas HTTP

Los endpoints destinados a la creación de recursos fueron modificados para devolver respuestas `ResponseEntity` con el estado HTTP 201 Created. Además, se incorporó el encabezado `Location`, el cual contiene la dirección del recurso recién creado. Esta implementación reemplaza las respuestas anteriores con código 200 OK y permite mantener un comportamiento más consistente con las buenas prácticas de las API REST. El endpoint de autenticación conserva la respuesta 200 OK, debido a que su función es validar credenciales y generar un token, no crear un recurso persistente.

Manejo global de excepciones

Se implementó un mecanismo centralizado para el tratamiento de errores mediante la clase `GlobalExceptionHandler`, utilizando la anotación `@RestControllerAdvice`. Esta implementación permite que todas las excepciones sean respondidas utilizando una estructura JSON uniforme, facilitando la interpretación de errores por parte de los consumidores de la API.

Para ello se incorporaron las clases:

- `ApiErrorResponse`
- `ResourceNotFoundException`
- `GlobalExceptionHandler`

Además, se configuró el manejo de excepciones para respuestas 400, 401, 404 y 500, reemplazando respuestas vacías por mensajes descriptivos y consistentes.

Validación de la documentación

Una vez implementada la documentación, se verificó su funcionamiento utilizando Swagger UI y Postman.

Además, se exportó el documento OpenAPI desde el endpoint `/v3/api-docs`, comprobando que la especificación generada coincidiera con la implementación del backend.

Exportación del documento OpenAPI desde el endpoint `/v3/api-docs`.

En esta evidencia se observa el archivo JSON generado automáticamente desde el endpoint `/v3/api-docs`, el cual contiene la especificación completa de la API conforme al estándar OpenAPI.



The screenshot shows a web browser window with the address bar displaying `localhost:8081/v3/api-docs`. Below the address bar, there is a button labeled "Dar formato al texto". The main content area displays a JSON document representing the OpenAPI specification for the "Minimarket Plus API".

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "Minimarket Plus API",
    "description": "API REST para la gestión de productos, inventario, ventas, usuarios y carrito del sistema Minimarket Plus.",
    "contact": {
      "name": "Grupo 28",
      "email": "grupo28@duoc.cl"
    },
    "license": {
      "name": "Uso Académico"
    },
    "version": "1.0"
  },
  "servers": [
    {
      "url": "http://localhost:8081",
      "description": "Generated server url"
    }
  ],
  "tags": [
    {
      "name": "Carrito",
      "description": "Endpoints para la gestión del carrito de compras del sistema Minimarket Plus"
    },
    {
      "name": "Productos",
      "description": "Endpoints para la gestión de productos del sistema Minimarket Plus"
    },
    {
      "name": "Usuarios",
      "description": "Endpoints para la gestión de usuarios del sistema Minimarket Plus"
    }
  ],
  "paths": {
    "/api/usuarios/{id}": {
      "get": {
        "tags": [
          "Usuarios"
        ],
        "operationId": "obtenerUsuarioPorId",
        "parameters": [
          {
            "name": "id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "integer",
              "format": "int64"
            }
          }
        ],
        "responses": {
          "200": {
            "description": "OK",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/EntityModelUsuarioDto"
                }
              }
            }
          }
        }
      },
      "put": {
        "tags": [
          "Usuarios"
        ],
        "operationId": "actualizarUsuario",
        "parameters": [
          {
            "name": "id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "integer",
              "format": "int64"
            }
          }
        ]
      }
    }
  }
}
```


Importación del documento OpenAPI en Postman para validar los endpoints documentados.

La especificación OpenAPI fue exportada e importada correctamente en Postman, permitiendo validar que la documentación generada representa de forma precisa los endpoints implementados en el sistema.



Integración del backend

Una vez implementados los distintos componentes del proyecto, se realizaron pruebas funcionales para verificar su funcionamiento conjunto. Estas validaciones permitieron comprobar la correcta integración entre los controladores, la capa de servicios, los repositorios, la base de datos y los mecanismos de seguridad implementados.

Durante las pruebas se confirmó que el proceso de autenticación mediante JWT funciona correctamente, permitiendo que únicamente los usuarios autenticados accedan a los recursos protegidos. Asimismo, las reglas de autorización definidas en Spring Security restringen el acceso a los distintos endpoints según el rol asignado a cada usuario, garantizando un control adecuado sobre las operaciones disponibles.

También se verificó la integración de la documentación generada mediante OpenAPI con los servicios implementados, permitiendo consultar y probar los endpoints desde Swagger UI. Del mismo modo, las respuestas enriquecidas con HATEOAS incorporan correctamente los enlaces dinámicos entre recursos, facilitando la navegación dentro de la API.

Como parte de la integración final del backend, también se incorporó un manejo global de excepciones mediante `@RestControllerAdvice`, permitiendo que todas las respuestas de error utilicen un formato uniforme independientemente del controlador que origine la excepción.

Finalmente, la ejecución de las pruebas unitarias permitió comprobar que las principales funcionalidades del sistema operan correctamente y que las modificaciones realizadas durante el desarrollo no afectaron el comportamiento esperado de la aplicación. En conjunto, estas validaciones demuestran que los distintos componentes del backend funcionan de manera integrada, proporcionando una solución estable, segura y alineada con los requerimientos del proyecto.

Mejoras implementadas durante el desarrollo

A lo largo del desarrollo del proyecto MiniMarket Plus se realizaron diversas mejoras con el objetivo de fortalecer la seguridad, la calidad del código y la mantenibilidad de la aplicación.

Una de las principales mejoras fue la incorporación de Spring Security junto con autenticación basada en JSON Web Token (JWT), permitiendo proteger los recursos de la API y controlar el acceso mediante los roles Administrador, Empleado y Cliente. Además, las contraseñas de los usuarios fueron almacenadas utilizando BCrypt, incrementando la seguridad de las credenciales.

Con el propósito de mejorar la calidad del software, se desarrolló un conjunto de pruebas unitarias utilizando JUnit 5, Mockito y JaCoCo, permitiendo validar la lógica de negocio de los principales servicios y medir la cobertura del código mediante reportes automatizados.

También se incorporó la documentación automática de la API mediante OpenAPI (Swagger), facilitando la consulta de los endpoints, parámetros y respuestas disponibles. Como complemento, se implementó Spring HATEOAS, agregando enlaces dinámicos a las respuestas JSON para mejorar la navegación entre recursos y facilitar la integración con aplicaciones cliente.

Finalmente, se realizaron ajustes sobre la estructura del proyecto y la configuración de seguridad, unificando la gestión de roles, incorporando validaciones de datos y manteniendo una arquitectura organizada basada en controladores, servicios, repositorios y entidades. Estas mejoras permitieron obtener un backend más seguro, mantenible y preparado para futuras ampliaciones.

Conclusión

El desarrollo del proyecto MiniMarket Plus permitió integrar los principales contenidos abordados durante la asignatura, implementando un backend basado en Spring Boot con una arquitectura organizada y preparada para futuras ampliaciones.

Durante el desarrollo se implementaron mecanismos de autenticación y autorización utilizando Spring Security y JWT, permitiendo proteger los recursos de la aplicación mediante control de acceso basado en roles. Asimismo, se desarrollaron pruebas unitarias utilizando JUnit 5, Mockito y JaCoCo, validando el correcto funcionamiento de los principales componentes del sistema y contribuyendo a mejorar su calidad y confiabilidad.

También se incorporó documentación automática mediante OpenAPI (Swagger) e implementación de Spring HATEOAS, facilitando la comprensión y el consumo de la API por parte de otros desarrolladores.

En conjunto, las funcionalidades implementadas y las pruebas realizadas permitieron comprobar el correcto funcionamiento e integración de los distintos componentes del backend, obteniendo una solución estable, segura y alineada con los requerimientos definidos para el proyecto MiniMarket Plus.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.